

인공지능

Artificial Intelligence

딥러닝과 파이토치

MLP→DNN

5.1 딥러닝의 등장

- 딥러닝의 등장이 늦어진 이유
 - 깊은 신경망(딥러닝)에 대한 아이디어는 다층 퍼셉트론(MLP)이 등장해 혁신을 이루던 1980년대에 이미 나왔었음.
 - 다층 퍼셉트론에 은닉층을 여러 개 추가하면 깊은 신경망이 되기 때문에 누구나 쉽게 생각할 수 있었으나, 은닉층을 추가해 신경망을 깊게 만들면 제대로 학습되지 않는 문제가 있었음.
- 깊은 신경망 학습에 번번이 실패하는 이유
 - 1) 오류 역전파 알고리즘은 출력층에서 시작해 입력층 방향으로 진행하면서 그레디언트 미분값을 계산하고 가중치를 갱신하는데, 단계적으로 여러 층을 거치면서 그레디언트 값이 점점 작아져 입력층에 가까워지면 변화가 거의 없는 **그레디언트 소멸 문제** `gradient vanishing` 가 발생함
 - 2) 훈련 집합의 크기는 작은 상태로 머물러 있는데 추정할 매개변수는 크게 늘어 **과잉 적합** `over-fitting`에 빠질 위험이 더욱 커짐
 - 3) 학습이 어려운 또 다른 이유는 **과다한 계산 시간**임. 예전에는 병렬 처리를 하려면 값비싼 슈퍼컴퓨터를 사용하는 수밖에 없었음. (GPU의 등장으로 해당 문제 해결)

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 요인
 - 딥러닝의 등장 및 부상에 새롭게 제안된 이론이나 원리는 별로 없음
 - 신경망의 기본 구조와 동작, 학습 알고리즘의 기본 원리는 동일함
 - **저렴한 GPU 등장**
 - 성능이 뛰어나고 상대적으로 가격이 저렴한 GPU가 등장하면서, 대학 실험실에서도 손쉽게 병렬 처리 연산을 할 수 있게 되었음
 - 학습 시간이 10~100배 단축으로 다양한 실험을 시도 할 수 있게 되었음
 - 예들 들어, 딥러닝 모델 하나를 학습하는데 이틀이 걸린다면 200개의 서로 다른 하이퍼 매개변수 조합 중 최적을 선택하는 작업은 1년 이상 걸리지만 GPU를 사용하면 일주일 이내에 마칠 수 있음

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 요인
 - 인터넷의 발달로 학습 데이터가 크게 증가
 - 예를 들어, ImageNet은 인터넷에서 수집한 1400만 장 정도의 자연 영상을 카테고리 별로 분류해 제공함
 - 게다가, 데이터에 가우시안 잡음을 섞거나 영상을 조금 이동하거나 회전해 데이터를 인위적으로 수십~수백 배 증가시키는 기법이 개발되었음

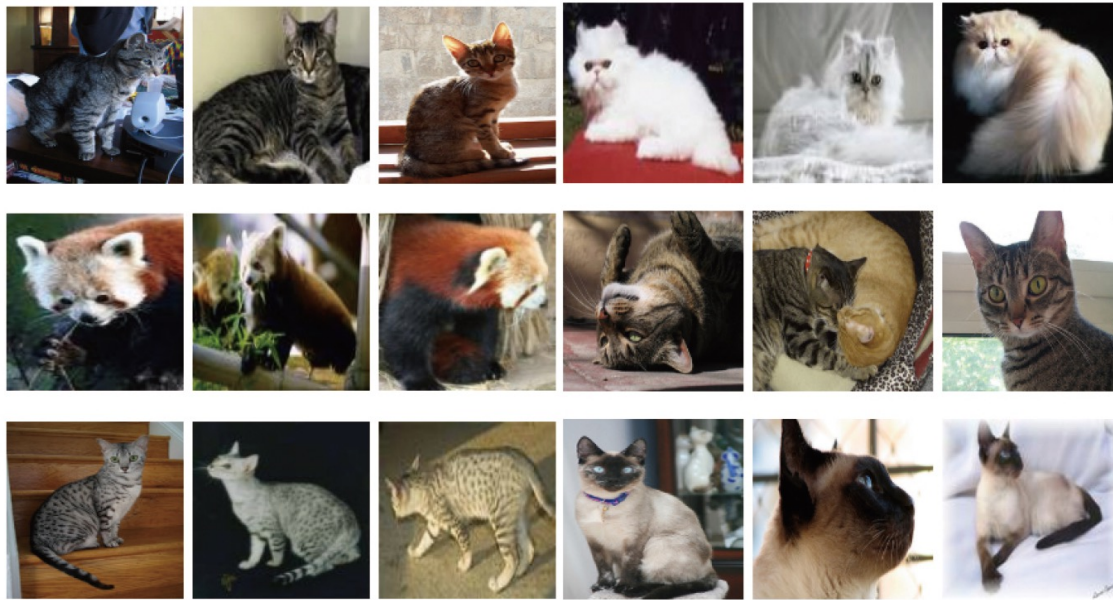


그림 5-2 자연 영상의 심한 변화 예(ImageNet의 고양이 부류 사진)

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 요인

- 인터넷의 발달로 학습 데이터가 크게 증가. (완전 다른 이야기! 최근 트렌드)

Generative Data Refinement (GDR) : 제안자→Google DeepMind 연구진 (2025년 초 공개)

- 문제정의:

- LLM 학습용 데이터가 점점 고갈됨 → 더 이상 “새로운 인간 생성 텍스트” 확보가 어렵다.
 - 기존에 수집된 대규모 데이터 중 상당수는 부적합 (toxicity, privacy violation, fact error, 품질 저하 등) 때문에 학습에서 제외하고 있었음.
 - 결과적으로 usable 데이터 풀(pool)이 급속히 줄어들음.

- 아이디어:

- 기존 unusable 데이터를 generative 모델로 다시 쓰게 함
 - 단순히 필터링으로 버리는 대신, 생성형 모델을 활용해 문장을 고쳐서 유해성/오류를 줄이고 학습 가능한 high-quality 데이터로 변환.
 - 예시: 욕설이나 개인정보가 들어간 문장을 “깨끗한 표현”으로 재작성, fact error를 제거하거나 부드러운 표현으로 바꾸기.

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 요인
 - 학습을 효과적으로 실행할 수 있는 다양한 알고리즘 개발
 - 계산은 단순하고 성능은 더 좋은 활성화 함수^{activation function} 를 발견
 - 특히 ReLU의 발견으로 모델의 그래디언트 소멸 문제가 크게 완화되었음
 - 학습에 효과적인 다양한 규제 기법^{regularization} 이 개발되었음
 - 가중치를 작은 값으로 유지하는 가중치 축소^{weight decay} 기법
 - 임의로 일정 비율의 노드를 선택해 불능으로 놓고 학습하는 드롭아웃^{dropout} 기법
 - 조기 멈춤, 데이터 확대, 앙상블 등의 다양한 규제 기법이 개발되었음
 - 다양한 손실 함수와 최적화 알고리즘^{optimizer} 개발도 성능 향상에 크게 기여하였음

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 사례 (학술적인 측면)
 - **컨볼루션 신경망(CNN)이 딥러닝의 가능성을 열었음**
 - 컨볼루션 신경망은 특징 추출에 적합한 작은 크기의 컨볼루션 마스크를 사용하기 때문에 완전연결 구조인 다층 퍼셉트론 보다 매개변수가 훨씬 적지만 훨씬 우수한 특징을 추출함
 - 이런 이유로 컨볼루션 신경망에서 우수한 성능이 가장 먼저 입증되었음
 - 1990년대에 뉴욕 대학교의 리쿤 교수는 CNN으로 필기 숫자 인식에서 획기적인 성능 향상을 얻었고, 이 기술혁신으로 필기 숫자 인식이 수표 인식에서 실용화 수준에 도달하게 됨(1998)
 - 이후 CNN에서 개발된 여러 기법은 딥러닝 전반에 큰 영향을 미침

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 사례 (학술적인 측면)
 - **컨볼루션 신경망(CNN)이 딥러닝의 가능성을 열었음**
 - 르쿤이 필기 숫자에서 CNN의 가능성을 보였음에도 불구하고 자연 영상에 대한 시도는 소극적이었음
 - 2012년 ILSVRC 에서 CNN을 사용한 AlexNet이 오류율 15.3%라는 경이로운 성능으로 우승함. 전년도에 고전적인 기계 학습을 사용한 팀이 오류율 25.8%로 우승한 사실을 감안하면 CNN의 가능성을 보여주기에 충분함
 - 이때부터 컴퓨터 비전 연구는 고전적인 기법에서 딥러닝으로 패러다임 전환이 이루어짐
 - **딥러닝은 음성인식 분야의 혁신을 가져옴**
 - 딥러닝 이전에는 음성 인식을 주로 통계적 기법과 HMM^{Hidden Markov Model}을 이용해 구현하였음
 - 2009년경에 토론토 대학교의 힌튼 교수 연구팀은 음성 인식에 딥러닝을 적용하기 시작함
 - 힌튼 교수는 안드로이드 운영체제에 들어가는 음성 인식 소프트웨어의 단어 인식 오류율을 단숨에 25%나 줄였음

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 사례 (학술적인 측면)
 - Transformer 를 통한 LLM 시대 개막

항목	CNN	Transformer
시기	2012 (AlexNet)	2017 (Attention is All You Need)
혁신 포인트	이미지에서 feature learning 혁신	순차 데이터 처리 혁신 (RNN 종식)
상징적 의미	딥러닝이 기존 패러다임을 압도	Foundation Model·LLM 시대 개막
파급력	CV 전분야 표준화	NLP → CV/ 멀티모달 → 로봇틱스까지 확장

5.1.1 딥러닝의 기술 혁신

- 딥러닝의 기술 혁신 사례 (학술적인 측면)
 - ViT (Vision Transformer)는 컴퓨터 비전의 Foundation Model 시대의 문을 열었음

항목	CNN (2012 이후)	ViT (2020 이후)
혁신 포인트	딥러닝이 전통 기법을 압도함	CV에서도 Transformer가 가능함을 입증
효과	“딥러닝 시대” 개막	“모달리티 융합 시대” 가속
파급력	이미지 분류·검출· 세그멘테이션의 표준	멀티모달 학습·VLM· Foundation Model 표준

5.1.2 딥러닝 소프트웨어

- 여러 대학과 기업은 딥러닝 소프트웨어를 개발해 내부 연구 그룹끼리 공유하기 시작함
- 공유를 통해 딥러닝 소프트웨어는 점점 개선되어 큰 규모로 발전했고, 개발자들은 누구나 사용할 수 있도록 라이브러리 형태로 제작해 경쟁적으로 공개함
- 딥러닝 소프트웨어 자체는 대부분 C와 C++ 로 개발하지만, 소프트웨어를 불러 쓰는 인터페이스 언어로는 파이썬과 같은 스크립트 언어를 채택함

5.1.2 딥러닝 소프트웨어

표 5-1 딥러닝 소프트웨어

이름	개발 그룹	최초 공개일	작성 언어	인터페이스 언어	전이학습 지원	철저한 관리
씨아노(Theano)	몬트리올 대학교	2007년	파이썬	파이썬	○	X
카페(Caffe)	UC버클리	2013년	C++	파이썬, 매트랩, C++	○	X
텐서플로 (TensorFlow)	구글 브레인	2015년	C++, 파이썬, CUDA	파이썬, C++, 자바, 자바스크립트, R, Julia, Swift, Go	○	○
케라스(Keras)	프랑소와 솔레 (François Chollet)	2015년	파이썬	파이썬, R	○	○
파이토치(PyTorch)	페이스북	2016년	C++, 파이썬, CUDA	파이썬, C++	○	○

5.1.2 딥러닝 소프트웨어

- 구글 트렌드를 통해 텐서플로우와 파이토치의 최근 영향력을 비교한 그래프를 살펴보면, 2020년 12월 기준으로 둘의 영향력은 막상막하로 보임

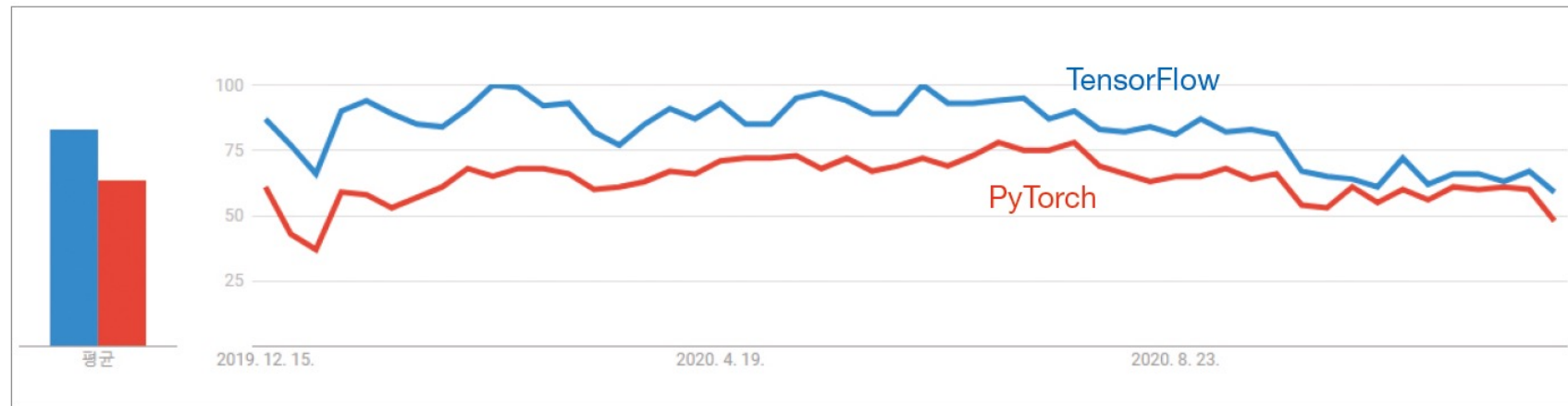


그림 5-3 구글 트렌드를 통해 비교한 텐서플로우와 파이토치의 영향력

5.1.2 딥러닝 소프트웨어

요약 타임라인

2008 - 2013: Theano, Caffe → 초기 연구자용 툴

2013 - 2016: Torch, Chainer → 동적 그래프 개념 도입

2015 - 2018: TensorFlow, PyTorch, MXNet → 양강 체제, 산업 vs 연구

2019 - 2022: PyTorch 사실상 표준, Hugging Face 허브 확산

2023 - Now: JAX, DeepSpeed, Megatron 등 초거대 모델·효율 중심

■ 통합기 (2019~2022): PyTorch 사실상 표준화

- PyTorch가 연구·산업계 모두에서 주도적 위치 확보.
- TensorFlow는 2.x로 전환하며 “Eager Execution” 추가 → PyTorch 스타일 따라감.
- ONNX (Open Neural Network Exchange, 2017~) → 프레임워크 간 호환성 표준.
- 특징: PyTorch 생태계 중심으로 연구자 커뮤니티 통합, 오픈소스 모델 허브(Hugging Face)가 폭발적으로 성장

5.1.2 딥러닝 소프트웨어

요약 타임라인

2008 - 2013: Theano, Caffe → 초기 연구자용 툴

2013 - 2016: Torch, Chainer → 동적 그래프 개념 도입

2015 - 2018: TensorFlow, PyTorch, MXNet → 양강 체제, 산업 vs 연구

2019 - 2022: PyTorch 사실상 표준, Hugging Face 허브 확산

2023 - Now: JAX, DeepSpeed, Megatron 등 초거대 모델·효율 중심

- 최신 동향 (2023~현재): Foundation Model & 효율 중심
 - **특징: 단순 프레임워크 경쟁 → 초대규모 모델 학습 생태계로 확장.**
 - JAX (Google, 2018~)
 - NumPy 호환 + XLA 컴파일 최적화.
 - Flax, Haiku 같은 딥러닝 라이브러리 등장.
 - TPU·대규모 모델 연구에서 각광.
 - DeepSpeed (Microsoft, 2020~)
 - 초거대 모델 학습 최적화 (ZeRO optimizer 등).
 - Megatron-LM (NVIDIA, 2019~)
 - GPT 계열 대규모 모델 학습용 프레임워크.

5.2 파이토치 개념 익히기

- 본 강의에서는 인공지능을 구현하기 위해 파이토치 라이브러리를 사용함
- 본 강의의 실험을 위해 파이토치를 자신의 컴퓨터에 직접 설치할 수 있으나, 본 강의에서는 환경 설정이 완료된 <캐글 노트북>을 사용할 예정임
- 즉, 개인 컴퓨터에 파이토치를 직접 설치하는 방법은 본 강의에서 다루지 않음

5.2 파이토치 개념 익히기

5.2.1 파이토치와 넘파이 호환

[프로그램 5-1] 파이토치 버전과 동작 확인

[2]:

```
1 import torch
```

```
2
```

```
3 print(torch.__version__)
```

03행 torch version 확인

```
4 a = torch.rand(2,3)
```

```
5 print(a)
```

```
6 print(type(a))
```

04행 0~1사이의 값을 가지는 난수를 원하는 크기에 해당하는 Tensor 반환



```
2.6.0+cu124
```

```
tensor([[0.5793, 0.0645, 0.8477],  
        [0.2953, 0.2734, 0.5942]])
```

```
<class 'torch.Tensor'>
```

<https://www.kaggle.com/code/yukyungchoi/2025-2-ai-w5-p1-codingnote>

5.2 파이토치 개념 익히기

5.2.1 파이토치와 넘파이이 호환

[프로그램 5-2] pytorch 텐서와 numpy 텐서의 호환 (사칙연산 가능)

▷

```
1 import torch
2 import numpy as np
3
4 t = torch.rand(2,3)
5 print('pytorch로 생성한 텐서 : \n {}'.format(t))
6
7 n = np.random.uniform(size=(2,3))
8 print('pytorch로 생성한 텐서 : \n {}'.format(n))
9
10 # pytorch 텐서와 numpy 텐서는 상호 호환이 가능함
11 res = t + n
12 print('\n덧셈 결과 : \n {}'.format(res))
13
```

04행 0~1사이의 값을 가지는 난수를 원하는 크기에 해당하는 Tensor 반환

07행 low~high사이의 랜덤 난수를 생성하여 원하는 사이즈의 ndarray 반환

pytorch로 생성한 텐서 :

```
tensor([[0.5047, 0.4212, 0.9179],
        [0.1722, 0.0924, 0.1931]])
```

pytorch로 생성한 텐서 :

```
[[0.0992473  0.03076871 0.48778488]
 [0.86071889 0.50314445 0.35460127]]
```

덧셈 결과 :

```
tensor([[0.6040, 0.4520, 1.4057],
        [1.0329, 0.5956, 0.5477]], dtype=torch.float64)
```

+ Code

+ Markdown

5.2 파이토치 개념 익히기

5.2.2 텐서 이해하기

[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

▷

```
1 import torch
2 from torchvision import datasets
3 from torchvision.transforms import ToTensor
4
5 MNIST_training_data = datasets.MNIST(
6     root='data',|
7     train=True,
8     download=True,
9     transform=ToTensor()
10 )
11
12 MNIST_test_data = datasets.MNIST(
13     root='data',
14     train=False,
15     download=True,
16     transform=ToTensor()
17 )
18
19 print("MNIST 학습 이미지 데이터 텐서 모양 : {}".format(MNIST_training_data[0][0].shape))
20 print("MNIST 학습 라벨 데이터 : {}".format(MNIST_training_data[0][1]))
21
22 print("MNIST 테스트 이미지 데이터 텐서 모양 : {}".format(MNIST_test_data[0][0].shape))
23 print("MNIST 테스트 라벨 데이터 : {}".format(MNIST_test_data[0][1]))
```

06행 root : data를 다운 받을 root 경로

07행 train : Train data를 받을지 말지 (True/False)

08행 download : 직접 다운로드 하여 root 경로에 저장할지 말지 (True/False)

09행 transform : 데이터를 불러올 때 어떤 변환을 적용할 지

```
MNIST 학습 이미지 데이터 텐서 모양 : torch.Size([1, 28, 28])
MNIST 학습 라벨 데이터 : 5
MNIST 테스트 이미지 데이터 텐서 모양 : torch.Size([1, 28, 28])
MNIST 테스트 라벨 데이터 : 7
```

5.2 파이토치 개념 익히기

5.2.2 텐서 이해하기

[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

```
1 import torch
2 from torchvision import datasets
3 from torchvision.transforms import ToTensor
```

[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

- MNIST_training_data / MNIST_test_data 는 Dataset 객체이고, 인덱스로 접근하면 (image_tensor, label) 형태의 튜플을 반환함
 - MNIST_training_data[0] → (image, label)
 - MNIST_training_data[0][0] → image (텐서)
 - MNIST_training_data[0][1] → label (정답, int)

```
12 MNIST_test_data = datasets.MNIST(
13     root='data'
14 )
15
16 # MNIST 학습 데이터 중 첫번째 데이터와 라벨
17 MNIST_training_data[0][0]
18 MNIST_training_data[0][1]
19
20 # MNIST 학습 데이터 중 마지막 데이터와 라벨
21 MNIST_training_data[60000-1][0]
22 MNIST_training_data[60000-1][1]
```

```
MNIST 학습 이미지 데이터 텐서 모양 : torch.Size([1, 28, 28])
MNIST 학습 라벨 데이터 : 5
MNIST 테스트 이미지 데이터 텐서 모양 : torch.Size([1, 28, 28])
MNIST 테스트 라벨 데이터 : 7
```

5.2 파이토치 개념 익히기

- 튜플 (tuple)의 정의
 - 여러 개의 값을 하나의 묶음으로 저장하는 자료형
 - 리스트(list)와 비슷하지만, 가장 큰 차이는 수정 불가능(immutable)
 - 소괄호 () 를 사용해서 만듦

◆ 기본 예시

python

```
t = (10, 20, 30)
print(t[0]) # 10
print(t[1]) # 20
```

- `t[0]` → 10
- `t[1]` → 20
- 튜플도 인덱스로 접근 가능.

◆ 리스트와의 차이

python

```
# 리스트
l = [1, 2, 3]
l[0] = 100
print(l) # [100, 2, 3] ✅ 수정 가능
```

```
# 튜플
t = (1, 2, 3)
# t[0] = 100 # ❌ 오류 발생 (TypeError)
```

즉, 리스트는 mutable(변경 가능), 튜플은 immutable(변경 불가).

5.2 파이토치 개념 익히기

- 튜플 (tuple) 의 특징
 - 순서가 있다 → 인덱싱, 슬라이싱 가능
 - 중복을 허용한다 → (1, 2, 2, 3) 가능
 - 수정 불가능하다 → 요소 추가, 삭제, 변경 불가
 - 다른 자료형 포함 가능
- 튜플 (tuple) 이 유용한 이유
 - 변하지 않아야 하는 데이터를 표현할 때 안전
 - 딕셔너리의 키로 사용할 수 있음 (리스트는 변경 가능해서 키로 못 씀)
 - 함수에서 여러 값을 한 번에 반환할 때 자주 사용됨

python

```
mixed = (1, "apple", [10,20], (3,4))
```

튜플 안에 리스트, 또 다른 튜플도 포함할 수 있다.

5.2 파이토치 개념 익히기

5.2.2 텐서 이해하기

[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

- **번외 : ToTensor() 변환이 해주는 일**
 - 데이터 타입 변환
 - PIL Image → PyTorch Tensor
 - dtype: torch.float32
 - 픽셀값 정규화
 - 원래 0~255 → 0.0~1.0 범위로 나눔 (/255)
 - 딥러닝에서 값이 작을수록 학습 안정성 높임 (Gradient 발산 방지)
 - 차원 순서 변경
 - 원래 이미지: (H, W, C)
 - PyTorch Tensor: (C, H, W)
 - 이유: PyTorch CNN 레이어는 (Batch, Channel, Height, Width) 구조를 요구함

5.2 파이토치 개념 익히기

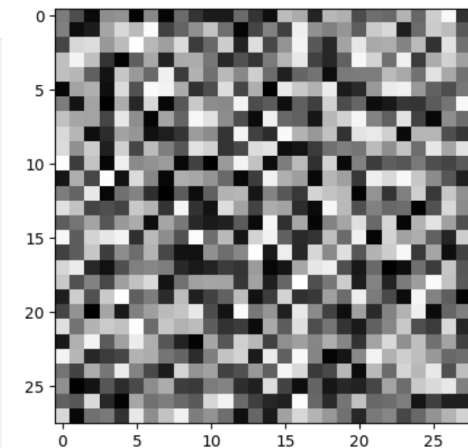
5.2.2 텐서 이해하기

[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

■ 번외: ToTensor() 변환이 해주는 일

```
> 1 from torchvision import transforms
  2 from PIL import Image
  3 import numpy as np
  4
  5 # 임의의 흑백 이미지 (28x28)
  6 arr = np.random.randint(0, 256, (28,28), dtype=np.uint8)
  7 img = Image.fromarray(arr) # PIL 이미지
  8
  9
 10 # 시각화
 11 import matplotlib.pyplot as plt
 12 plt.imshow(img, cmap="gray")
 13 plt.show()
 14
 15
 16 tensor = transforms.ToTensor()(img)
 17
 18 print("PIL 모드:", type(img), np.array(img).dtype, np.array(img).shape)
 19 print("Tensor 모드:", tensor.shape, tensor.dtype, tensor.min().item(), tensor.max().item())
 20
```

코드 실행 출력



PIL 모드: <class 'PIL.Image.Image'> uint8 (28, 28)
Tensor 모드: torch.Size([1, 28, 28]) torch.float32 0.0 1.0



5.2 파이토치 개념 익히기

5.2.2 텐서 이해하기

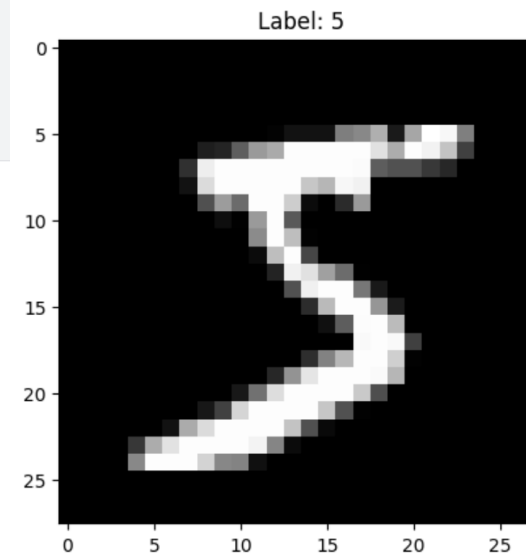
[프로그램 5-3] pytorch가 제공하는 데이터셋의 텐서 구조 확인하기

- Datasets객체인스턴스를 인덱싱하여 반환된 튜플은 아래와 같이 사용 가능

```
1 img, label = MNIST_training_data[0]
2
3 # 텐서 모양
4 print(img.shape)    # torch.Size([1, 28, 28]) -> 흑백채널, 높이, 너비
5 print(label)        # 정수 (0~9)
6
7 # 시각화
8 import matplotlib.pyplot as plt
9 plt.imshow(img.squeeze(), cmap="gray")
10 plt.title(f"Label: {label}")
11 plt.show()
```

코드 실행 출력

```
torch.Size([1, 28, 28])
5
```



5.3 파이토치 프로그래밍 기초

5.3.1 sklearn의 표현력 한계

- 4장에서는 sklearn 라이브러리를 활용해 기계학습 모델을 구현하였음
- MLP를 떠올려보면 **sklearn** 은 딥러닝을 구현하기에는 표현력의 한계가 있음
- 딥러닝 모델은 MLP 처럼 단순하지 않으며, 컨볼루션 연산을 사용하는 “**컨볼루션 신경망**”으로 확장되고, 은닉 노드끼리 에지로 연결되는 “**순환 신경망**”으로 확장되며, 신경망 구조를 벗어난 “**강화학습**”으로 확장됨
- <텐서플로우>나 <파이토치> 같은 라이브러리는 이런 복잡도를 지원할 수 있도록 바닥부터 **새롭게 설계한 딥러닝 전용 소프트웨어**이며, 이들은 명령어로 층을 쌓아가는 방식으로 프로그래밍을 진행함
- 딥러닝 프로그램에도 디자인 패턴이 있고, 디자인 패턴을 따른 좋은 예제 코드가 많으니 이들을 잘 관찰하고 기억하면 어렵지 않게 딥러닝 프로그래밍에 익숙해질 수 있음

sklearn library → pytorch library

5.3 파이토치 프로그래밍 기초

5.3.2 파이토치를 이용한 퍼셉트론 프로그래밍

[프로그램 5-4] 파이토치 프로그래밍 : 학습된 퍼셉트론 동작 (OR Operation, predefined Weight)

```
1 import torch
2
3 x = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
4 y = torch.tensor([-1], [1], [1], [1]])
5
6 w = torch.autograd.Variable(torch.tensor([[1.0], [1.0]]), requires_grad=True)
7 b = torch.autograd.Variable(torch.tensor([-0.5]), requires_grad=True))
8
9 s = x.matmul(w)+b
10 o = torch.sign(s)
11
12 print("[5-4프로그램] \n {}".format(o))
```

06행 gradient 추적을 허용하는 가중치 행렬 W 정의

07행 gradient 추적을 허용하는 편향 b 정의

10행 wx+b 형태로 계산한 다음 결과의 부호만을 반환



[5-4프로그램]

```
tensor([[ -1.],
        [ 1.],
        [ 1.],
        [ 1.]], grad_fn=<SignBackward0>)
```

5.3 파이토치 프로그래밍 기초

5.3.2 파이토치를 이용한 퍼셉트론 프로그래밍

[프로그램 5-5] 파이토치 프로그래밍 : 퍼셉트론 학습

```
1 import torch
2 def forward(w, b, x):
3     s = x.matmul(w)+b
4     o = torch.tanh(s) #은닉층 활성화함수 - 교재를 따라 tanh
5     return o
6
7 def loss_mes(pred, y):
8     loss = torch.mean((y-pred)**2)
9     return loss
10
11 if __name__ == '__main__':
12     x = torch.tensor([[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]])
13     y = torch.tensor([-1],[1],[1],[1]))
14
15     # 방식1 - 메모리 효율은 낮지만, 가독성 높음 (초보자용)
16     w = torch.tensor((torch.rand(2, 1) - 0.5), requires_grad=True)
17     b = torch.tensor([0.0], requires_grad=True)
18
19     # 방식2 - 메모리 효율 좋지만, 가독성 낮음
20     # w = (torch.rand(2, 1) - 0.5).clone().detach().requires_grad_(True)
21     # b = torch.zeros(1, requires_grad=True)
22
23     optimizer = torch.optim.SGD([w,b], lr=0.1)
24
25     for iter in range(500): # 0~499반복
26         pred = forward(w,b,x)
27         cost = loss_mes(pred, y)
28
29         optimizer.zero_grad()
30         cost.backward()
31         optimizer.step()
32         #print(iter)
33
34     pred = forward(w,b,x)
35     print(pred)
36     print(torch.sign(pred)) #출력층 활성화함수 : 계단함수
```

02행 $w \cdot x + b$ 형태로 계산하고 tanh 함수를 적용하여 순방향 연산 진행

07행 예측값과 정답값의 제곱 오차를 계산하는 MSE 손실함수

16-17행 (20-21행) gradient 추적을 허용하는 가중치,편향 정의

23행 확률적 경사하강법(SGD)에 해당하는 옵티마이저 선언(W,b를 최적화)

29행 `optimizer.zero_grad()` : 매 iteration 마다 gradient 초기화

30행 `cost.backward()` : 손실 함수로부터 역전파(backpropagation) 수행

31행 `optimizer.step()` : 역전파로부터 계산된 gradient를 토대로 W,b 업데이트

```
tensor([[ -1.],
        [  1.],
        [  1.],
        [  1.]])
```

5.3 파이토치 프로그래밍 기초

5.3.2 파이토치를 이용한 퍼셉트론 프로그래밍

[프로그램 5-6] 파이토치 프로그래밍 : 퍼셉트론 학습 (추상화)

```
1 import torch
2
3 # 클래스를 이용한 모델 정의, nn.Module 상속 받아 사용
4 class Perceptron(torch.nn.Module):
5     def __init__(self, input_dim, output_dim): #클래스 인스턴스 생성시 호출되는 생성자 개념
6         super(Perceptron, self).__init__()
7         self.Perceptron = torch.nn.Linear(in_features=input_dim,
8                                             out_features=output_dim,
9                                             bias=True)
10        self.activation = torch.nn.Tanh()
11        # 모델 이어 붙이는 부분
12        self.model = torch.nn.Sequential(self.Perceptron, self.activation)
13
14    def forward(self, x):
15        return self.model(x)
16
17
18 def loss_mes(pred, y):
19     loss = torch.mean((y-pred)**2)
20     return loss
21
22 if __name__ == '__main__':
23     x = torch.tensor([[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]])
24     y = torch.tensor([[1],[1],[1],[1]])
25
26     # 파이토치를 이용한 모델 직접 정의
27     Perceptron = Perceptron(input_dim=2, output_dim=1)
28     optimizer = torch.optim.SGD(Perceptron.parameters(), lr=0.1)
29
30     for iter in range(500): # 0~499반복
31
32         pred = Perceptron(x) #pred = Perceptron.forward(x) 무관
33         cost = loss_mes(pred, y)
34
35         optimizer.zero_grad()
36         cost.backward()
37         optimizer.step()
38         #print(iter)
39
40     pred = Perceptron(x) #pred = Perceptron.forward(x) 무관
41
42     print(pred)
43     print(torch.sign(pred))
```

01행 torch.nn.Module을 상속 받는 Class 정의

07행 torch.nn.Linear를 이용하여 선형 모델(퍼셉트론)을 정의

10행 활성화 함수는 torch.nn.Tanh()를 사용

12행 torch.nn.Sequential을 사용하여 모듈을 순차적으로 정의

14행 순전파를 다음과 같이 정의 (torch.nn.Module을 상속 받았기 때문)
Perceptron(x) 처럼 사용가능

35행 optimizer.zero_grad() : 매 iteration마다 gradient 초기화

36행 cost.backward() : 손실 함수로부터 역전파(backpropagation) 수행

37행 optimizer.step() : 역전파로부터 계산된 gradient를 토대로 W,b 업데이트

5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.1 MNIST 인식

[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식

```
1 import torch
2 import tqdm
3
4 from torchvision import datasets
5 from torchvision.transforms import ToTensor
6 |
7
8 class MultiLayerPerceptron(torch.nn.Module):
9     def __init__(self, input_dim, hidden_dim, output_dim):
10         super(MultiLayerPerceptron, self).__init__()
11         # MLP 각 레이어 정의
12         self.input_layer = torch.nn.Linear(in_features=input_dim, out_features=hidden_dim, bias=True)
13         self.hidden_layer = torch.nn.Linear(in_features=hidden_dim, out_features=output_dim, bias=True)
14         self.activation = torch.nn.Tanh()
15         # 모델 정의 : 레이어 연결
16         self.model = torch.nn.Sequential(self.input_layer, self.activation, self.hidden_layer)
17
18     def forward(self, x):
19         return self.model(x)
20
```


5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.1 MNIST 인식

[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식

```
21 if __name__ == '__main__':
22
23     # 로그용 딕셔너리 정의
24     log_dict = dict()
25     log_dict['train_acc'] = list()
26     log_dict['test_acc'] = list()
27     log_dict['train_loss'] = list()
28     log_dict['test_loss'] = list()
29
30     ## 데이터셋 준비 - W5P1에서 배움
31     MNIST_training_data = datasets.MNIST(root='data', train=True, download=True, transform=ToTensor())
32     MNIST_test_data = datasets.MNIST(root='data', train=False, download=True, transform=ToTensor())
33
34     ## 배치 단위 데이터셋 준비
35     train_dataloader = torch.utils.data.DataLoader(MNIST_training_data, batch_size=128)
36     test_dataloader = torch.utils.data.DataLoader(MNIST_test_data, batch_size=128, shuffle=False)
37
38     # 모델 정의 - W5P2 Perceptron 확장
39     MLP = MultiLayerPerceptron(input_dim=784, hidden_dim=1024, output_dim=10)
40     MLP = MLP.cuda()
41     #optimizer = torch.optim.Adam(MLP.parameters(), lr=0.001)
42     optimizer = torch.optim.SGD(MLP.parameters(), lr=0.001)
43     loss_mse = torch.nn.CrossEntropyLoss()
44
```

→ 학습 중간에 정확도와 손실을 기록할 딕셔너리 정의

5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.1 MNIST 인식

[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식

```
46 # 모델 학습
47 for iter in tqdm.tqdm(range(30)):
48     #MLP, history = train_epoch(train_dataloader, optimizer, MLP, loss_mse, history)
49     acc_train = 0
50     loss_train = 0
51     MLP.train()
52
53     for X, y in train_dataloader: #학습 데이터 배치단위 로드
54         X = X.view(-1, 784).cuda()
55         y = y.cuda()
56
57         pred = MLP(X) # 모델 forward
58         cost = loss_mse(pred, y) # 손실함수 계산
59
60         optimizer.zero_grad() # 기울기 초기화 필요 (파이토치가 기울기를 축적하여 사용하다보니..)
61         cost.backward() # 역전파 계산
62         optimizer.step() # 가중치 갱신
63
64         acc_train += torch.sum(torch.argmax(pred, dim=1)==y).item()
65         loss_train += cost.item()
66
67 # 학습 로그 기록
68 log_dict['train_acc'].append(acc_train/len(train_dataloader.dataset))
69 log_dict['train_loss'].append(loss_train)
70
```

→ model.train() : 모델을 학습 모드로 변경 (batchnorm이나 dropout 관련)

→ 28x28 크기의 데이터(X)를 784x1 데이터로 reshape & GPU 메모리에 올림

→ 정답 데이터(y)도 GPU 메모리에 올려줌

→ optimizer.zero_grad() : 매 iteration마다 gradient 초기화

→ cost.backward() : 손실 함수로부터 역전파(backpropagation) 수행

→ optimizer.step() : 역전파로부터 계산된 gradient를 토대로 W,b 업데이트

5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.1 MNIST 인식

[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식

```
71
72     # 모델 테스트
73     with torch.no_grad():
74         #MLP, history = test_epoch(test_dataloader, optimizer, MLP, loss_mse, history)
75         acc_test = 0
76         loss_test = 0
77         MLP.eval()
78
79     for X, y in test_dataloader: #테스트 데이터 배치단위 받기
80         X = X.view(-1, 784).cuda()
81         y = y.cuda()
82
83         pred = MLP(X)
84         cost = loss_mse(pred,y)
85
86         # 학습 단계가 아니므로, 아래의 단계 생략
87         #optimizer.zero_grad()
88         #cost.backward()
89         #optimizer.step()
90
91         acc_test += torch.sum(torch.argmax(pred, dim=1)==y).item()
92         loss_test += cost.item()
93
94     log_dict['test_acc'].append(acc_test/len(test_dataloader.dataset))
95     log_dict['test_loss'].append(loss_test)
96
```

→ model.eval() : 모델을 평가 모드로 변경 (batchnorm이나 dropout 관련)

→ 28x28 크기의 데이터를 784x1 데이터로 reshape & 동시에 GPU 메모리에 올림

→ 정답 데이터도 GPU 메모리에 올림

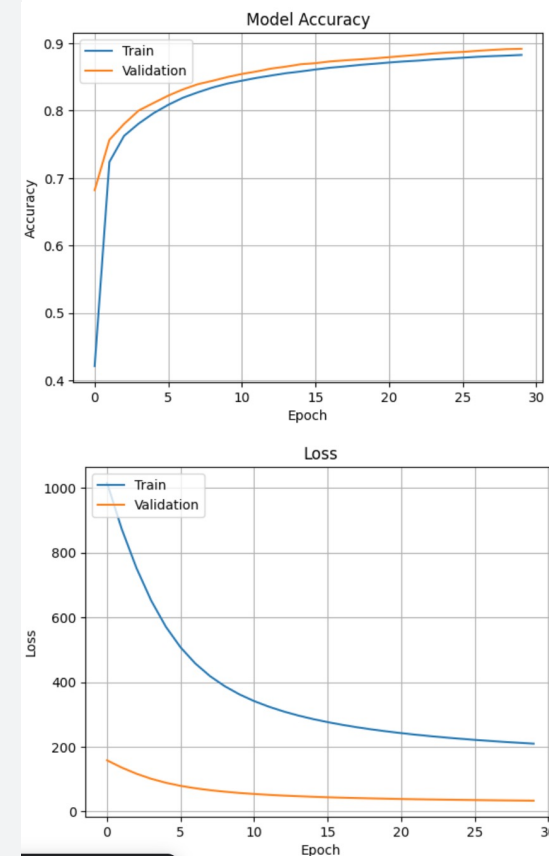
예측값과 정답값이 동일한 개수를 카운트하여 저장

5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.2 학습 곡선 시각화

[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식

```
1
2 import matplotlib.pyplot as plt
3
4 # 정확도 도식화
5 plt.plot(log_dict['train_acc'])
6 plt.plot(log_dict['test_acc'])
7 plt.title("Model Accuracy")
8 plt.xlabel("Epoch")
9 plt.ylabel("Accuracy")
10 plt.legend(['Train', 'Validation'], loc='upper left')
11 plt.grid()
12 plt.show()
13
14 # 손실 도식화
15 plt.plot(log_dict['train_loss'])
16 plt.plot(log_dict['test_loss'])
17 plt.title("Loss")
18 plt.xlabel("Epoch")
19 plt.ylabel("Loss")
20 plt.legend(['Train', 'Validation'], loc='upper left')
21 plt.grid()
22 plt.show()
```



과적합 발생 여부 확인을 위한 학습 곡선 시각화

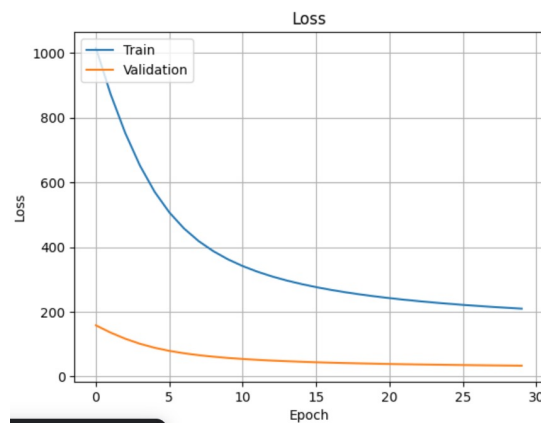
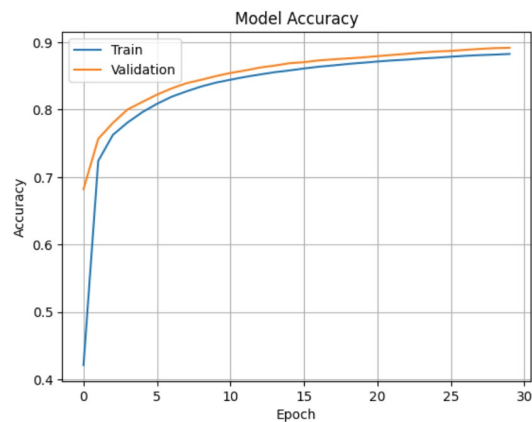
5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.1 MNIST 인식

[프로그램 5-7(b)] 다층 퍼셉트론으로 MNIST 인식

“[프로그램 5-7(a)] 다층 퍼셉트론으로 MNIST 인식” 코드를 좀 더 추상화, 일반화 한 코드

- 옵티마이저를 SGD, Adam으로 선택하여 비교하여 시각화



5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.3 fashion MNIST 인식

- 파이토치는 필기 숫자 데이터인 MNIST와 아주 비슷한 **fashion MNIST**를 제공함
- 샘플은 28x28 맵으로 표현되며 훈련 집합 60,000개 샘플, 테스트 집합 10,000개 샘플로 구성됨
- 라벨이 {0,1,2,3,4,5,6,7,8,9}에서 {T-shirt, top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot}로 바뀌고 **샘플 영상이 패션 아이템**이라는 점만 다름



5.4 파이토치 다층 퍼셉트론 프로그래밍

5.4.3 fashion MNIST 인식

[프로그램 5-8] 다층 퍼셉트론으로 Fashion MNIST 인식

[프로그램 5-8]은 [프로그램 5-7] 에서 “데이터셋 준비”하는 코드만 변경하면 됨

```
import torch
import tqdm

if __name__ == '__main__':

    history = dict()
    history['train_acc'] = list()
    history['test_acc'] = list()
    history['train_loss'] = list()
    history['test_loss'] = list()

    training_data = datasets.FashionMNIST(
        root="data",
        train=True,
        download=True,
        transform=ToTensor()
    )

    test_data = datasets.FashionMNIST(
        root="data",
        train=False,
        download=True,
        transform=ToTensor()
    )
```

12행 FashionMNIST 학습 데이터 정의

19행 FashionMNIST 평가 데이터 정의

다음 수업 시간

다음 시간에는 MNIST, Fashion MNIST 실습 진행 예정

Kaggle에서 Pytorch MLP 로 문제 해결하기

5.5 깊은 다층 퍼셉트론

- 〈다층 퍼셉트론〉에 은닉층을 더 많이 추가하면 〈깊은 다층 퍼셉트론〉 DMLP: Deep Multi-Layer Perceptron 이 되며, 깊은 다층 퍼셉트론은 은닉층만 추가하면 되기 때문에 가장 쉽게 생각할 수 있는 딥러닝 모델임

5.5 깊은 다층 퍼셉트론

5.5.1 구조와 동작

- 다층 퍼셉트론의 훈련집합 전체에 대한 동작 표현
- L-1번째 층과 L번째 층을 연결하는 가중치 행렬

$$\mathbf{O} = \tau_L \left(\cdots \tau_2 \left(\mathbf{U}^2 \tau_1 \left(\mathbf{U}^1 \mathbf{X}^T \right) \right) \right) \quad (5.5)$$

$$\mathbf{U}^l = \begin{pmatrix} u_{10}^l & u_{11}^l & \cdots & u_{1n_{l-1}}^l \\ u_{20}^l & u_{21}^l & \cdots & u_{2n_{l-1}}^l \\ \vdots & \vdots & \ddots & \vdots \\ u_{n_l 0}^l & u_{n_l 1}^l & \cdots & u_{n_l n_{l-1}}^l \end{pmatrix}, \quad l = 1, 2, \dots, L \quad (5.1)$$

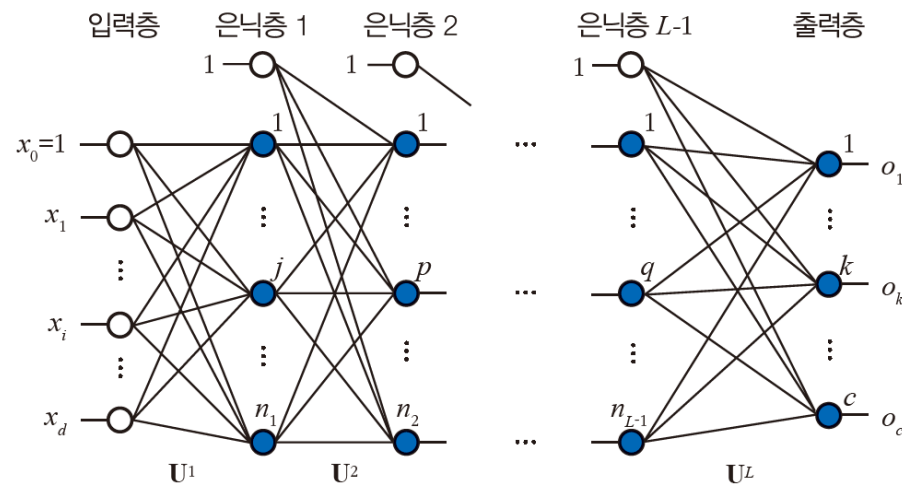


그림 5-8 깊은 다층 퍼셉트론의 구조

5.5 깊은 다층 퍼셉트론

5.5.1 구조와 동작

- [그림5-8]은 깊은 다층 퍼셉트론의 구조임
- 양 끝에 입력층과 출력층이 있으며 중간에 **L-1개의 은닉층이 있어 총 L개 은닉층 수+출력층의 층이 있음**
- 은닉층이 하나인 다층 퍼셉트론은 L=2인 특수한 경우임
- 입력층 특징 벡터의 요소 개수에 따라 d개 노드를 가지며 출력층은 부류 개수에 따라 c개 노드를 가짐
 - 예를 들어, MNIST 필기 숫자 데이터의 경우 화소를 특징으로 사용한다면 $d=28 \times 28=784$ 이고, $c=10$ 임, i번째 은닉층의 개수는 프로그래머가 설정하는 하이퍼 매개변수이며 n_i 로 표기함
- 깊은 다층 퍼셉트론은 인접한 두 층의 모든 노드 사이에 에지가 있는 **완전 연결 fc: fully connected 구조임**

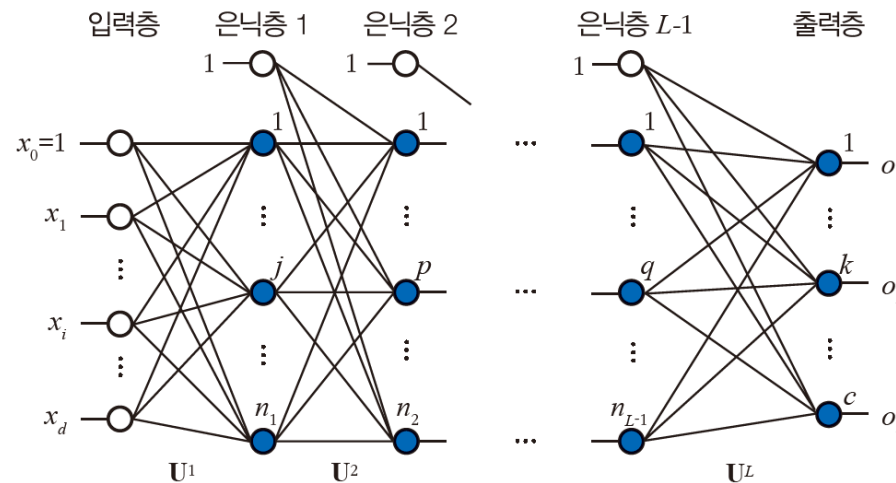


그림 5-8 깊은 다층 퍼셉트론의 구조

5.5 깊은 다층 퍼셉트론

5.5.1 구조와 동작

- 깊은 다층 퍼셉트론은 가중치가 많음
- 가중치가 많으면 1)계산 시간이 길어지고 2)과잉 적합이 발생할 가능성도 커짐
- 제시된 문제가 극복되어야 딥러닝(깊은 신경망 학습)이 성공할 수 있음
- 시간이 오래 걸리는 계산 문제는 GPU를 사용해 해결이 가능하고, 과잉 적합은 여러가지 규제 기법으로 해결 가능함
- MNIST 데이터셋 $L=5$, 은닉층 노드 개수가 500일 때의 총 가중치를 구해보자.
 - $L=5$ 이므로, 은닉층의 개수는 $L-1$ 로 4개임
 - 입력층-은닉층1 : $(784+1)*500$ $//+1$ 은 바이어스
 - 은닉층1-은닉층2 : $(500+1)*500*1$
 - 은닉층2-은닉층3: $(500+1)*500*1$
 - 은닉층3-은닉층4: $(500+1)*500*1$
 - 은닉층4-출력층 : $(500+1)*10$ $//10$ 개의 클래스
 - 따라서 총 가중치의 수는 1,149,010개임

5.5 깊은 다층 퍼셉트론

5.5.2 오류 역전파 알고리즘

- 깊은 다층 퍼셉트론을 학습하려면 **손실 함수** 정의와 손실 함수의 최저점을 찾는 **최적화 알고리즘**이 필요
- 손실 함수 : 미니 배치 단위(M)로 오차의 평균

$$J(\mathbf{U}^1, \mathbf{U}^2, \dots, \mathbf{U}^L) = \frac{1}{|M|} \sum_{\mathbf{x} \in M} \|\mathbf{y} - \mathbf{o}\|^2 \quad \leftarrow \text{평균제곱오차(MSE, mean squared error)}$$

$$= \frac{1}{|M|} \sum_{\mathbf{x} \in M} \|\mathbf{y} - \tau_L(\dots \tau_2(\mathbf{U}^2 \tau_1(\mathbf{U}^1 \mathbf{x}^T))\|^2 \quad (5.6)$$

$\leftarrow \tau$: 각 층의 활성화함수, $\mathbf{U}$: 각 층의 가중치

- 가중치 갱신 규칙 $\mathbf{U}^l = \mathbf{U}^l + \rho(-\nabla \mathbf{U}^l), l = L, L-1, \dots, 1 \quad (5.7)$

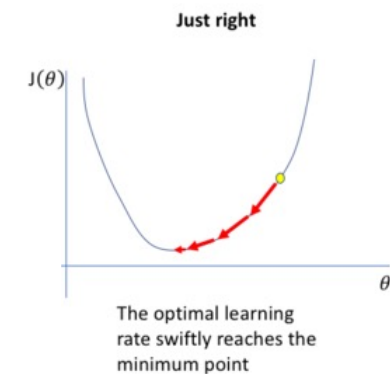
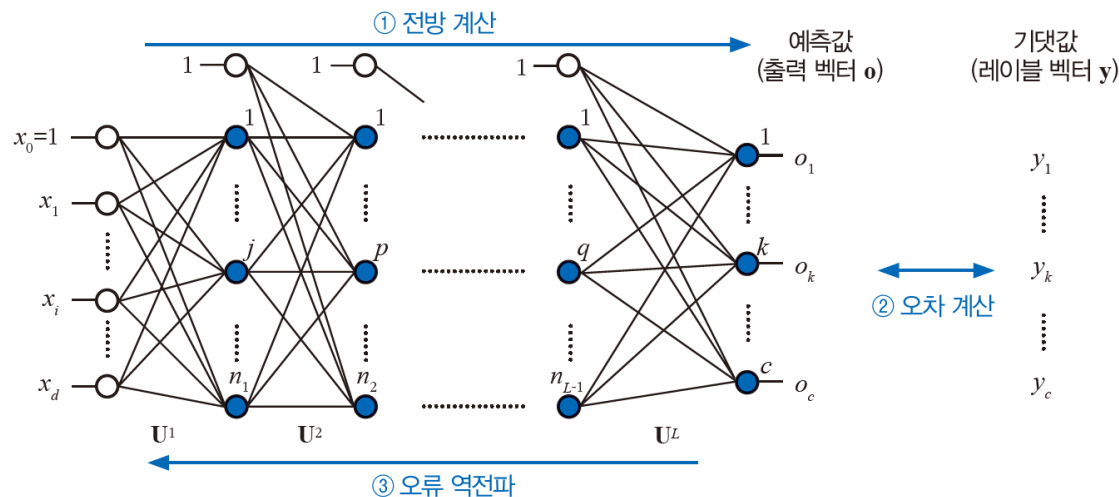


그림 5-9 깊은 다층 퍼셉트론이 사용하는 오류 역전파 알고리즘

5.5 깊은 다층 퍼셉트론

5.5.2 오류 역전파 알고리즘

- [그림5-9]는 다층 퍼셉트론의 학습 절차를 설명
 - 특징 벡터를 입력층에 입력
 - 전방 계산 (forward)
 - 전방 계산으로 얻은 예측 값을 기대 값과 비교해 오차 계산
 - 오차에 따라 $U^L, U^{L-1}, \dots, U^1$ 순으로 가중치 갱신

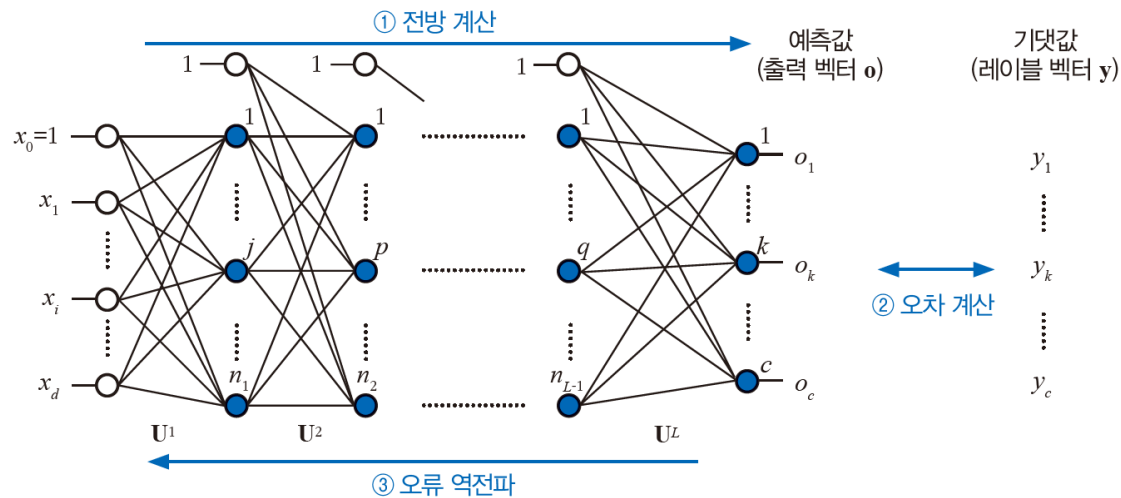


그림 5-9 깊은 다층 퍼셉트론이 사용하는 오류 역전파 알고리즘

5.5 깊은 다층 퍼셉트론

5.5.3 깊은 다층 퍼셉트론 프로그래밍

[프로그램 5-9] 깊은 다층 퍼셉트론으로 MNIST 인식 (프로그램 5-7 과 나머지 동일)

```
import torch

class DeepNeuralNetwork(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(DeepNeuralNetwork, self).__init__()

        self.input_layer = torch.nn.Linear(in_features=input_dim,
                                             out_features=hidden_dim, bias=True)
        self.hidden_layer1 = torch.nn.Linear(in_features=hidden_dim,
                                              out_features=hidden_dim, bias=True)
        self.hidden_layer2 = torch.nn.Linear(in_features=hidden_dim,
                                              out_features=hidden_dim, bias=True)
        self.hidden_layer3 = torch.nn.Linear(in_features=hidden_dim,
                                              out_features=hidden_dim, bias=True)
        self.output_layer = torch.nn.Linear(in_features=hidden_dim,
                                             out_features=output_dim, bias=True)

        self.activation = torch.nn.Tanh()

        self.model = torch.nn.Sequential(self.input_layer, self.activation,
                                          self.hidden_layer1, self.activation,
                                          self.hidden_layer2, self.activation,
                                          self.hidden_layer3, self.activation,
                                          self.output_layer, self.activation,)

    def forward(self, x):
        return self.model(x)
```

03행 torch.nn.Module을 상속 받는 Class 정의

07~16행 torch.nn.Linear를 이용하여 선형 모델(퍼셉트론)을 여러 개 정의

입력 : 784, 은닉층1 : 1024, 은닉층2 : 512, 은닉층3 : 512, 은닉층4 : 512, 출력층 : 10

09행 활성화 함수는 torch.nn.Tanh()를 사용

11행 torch.nn.Sequential을 사용하여 모듈을 순차적으로 정의

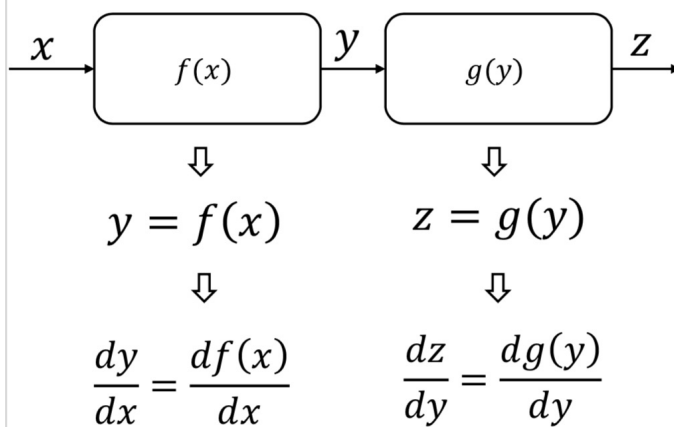
15행 순전파를 다음과 같이 정의 (torch.nn.Module을 상속 받았기 때문) Perceptron(x) 처럼 사용가능

5.6 딥러닝 학습 전략

5.6.1 그레디언트 소멸 문제와 해결책

- 미분 이론의 연쇄 법칙(chain rule)에 따르면,
[그림5-9]에서 l번째 층의 그레디언트는 오른쪽에 있는 l+1번째 층의 그레디언트에 자신의 층에서 발생한 그레디언트를 곱하여 구함

미분과 연쇄 법칙



연쇄 법칙 (Chain Rule)

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = \frac{dg(y)}{dy} \frac{df(x)}{dx}$$



합성 함수의 미분 (겉미분과 속미분)

$$\frac{dz}{dx} = \frac{dg(f(x))}{dx} = g'(f(x))f'(x)$$

연쇄 법칙을 이용한 미분의 계산. 고등학교 과정에서 배우는 **합성 함수의 미분**과 동일

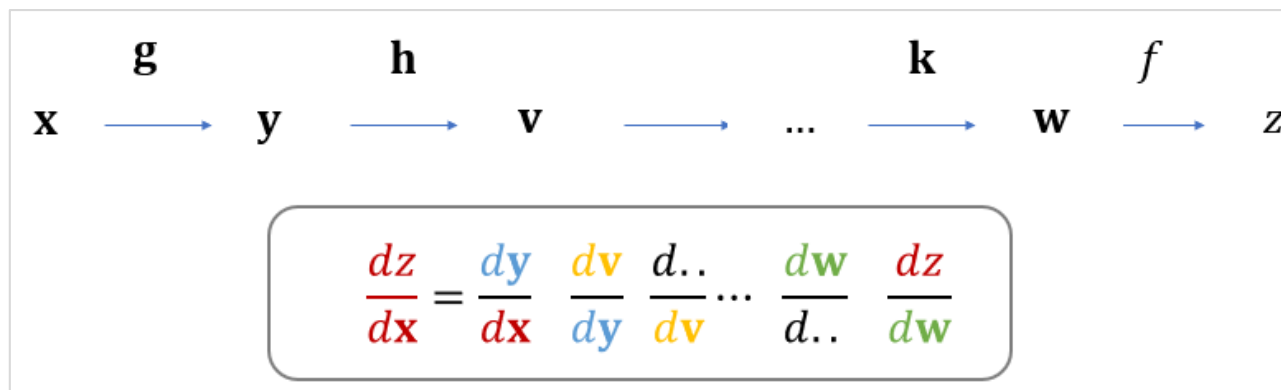
5.6 딥러닝 학습 전략

5.6.1 그레디언트 소멸 문제와 해결책

- 그레디언트가 작으면 오른쪽에서 왼쪽으로 진행하면서 그레디언트가 점점 작아지는 현상이 발생함
- 그레디언트가 기하급수적으로 작아지는 현상을 **그레디언트 소멸** gradient vanishing 이라 함
- 그레디언트 소멸 현상이 발생하면,

오른쪽에 있는 층의 가중치는 갱신이 제대로 일어나지만 왼쪽으로 갈수록 갱신이 매우 느림

결국 신경망 학습이 매우 느려져서 오랫동안 학습해도 수렴에 도달하지 못하는 문제가 발생함



5.6 딥러닝 학습 전략

5.6.1 그레디언트 소멸 문제와 해결책

- 미분 이론의 연쇄 법칙(chain rule)에 따르면, [그림5-9]에서 l번째 층의 그레디언트 오른쪽에 있는 l+1번째 층의 그레디언트에 자신의 층에서 발생한 그레디언트를 곱하여 구함
- 그레디언트가 작으면 오른쪽에서 왼쪽으로 진행하면서 그레디언트가 점점 작아지는 현상이 발생함
- 그레디언트가 기하급수적으로 작아지는 현상을 그레디언트 소멸 gradient vanishing 이라 함
- 그레디언트 소멸 현상이 발생하면 오른쪽에 있는 층의 가중치는 갱신이 제대로 일어나지만 왼쪽으로 갈수록 갱신이 매우 느림. 결국 전체적으로 신경망 학습이 매우 느려져서 오랫동안 학습해도 수렴에 도달하지 못하는 문제가 발생함
- 예) 깊이가 10, 모든 층의 그레디언트 0.01 이라고 가정
 - 열 번째 층의 그레디언트가 0.01인데,
 - 아홉 번째 층의 그레디언트는 $0.01 \times 0.01 = 0.0001$
 - 여덟 번째 층의 그레디언트는 $0.0001 \times 0.01 = 0.000001$
 - 일곱 번째 층의 그레디언트는 $0.000001 \times 0.01 = 0.00000001$
 - ...

5.6 딥러닝 학습 전략

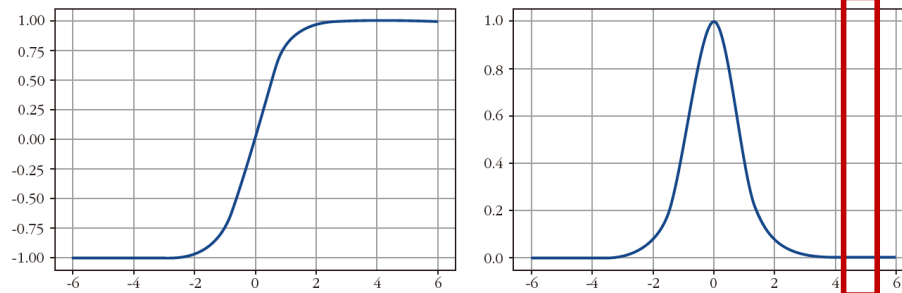
5.6.1.1 병렬 처리로 해결

- 그레디언트 소멸 문제에 대처하려면 여러 가지 방법을 결합해서 사용해야 함
- 가장 직접적인 방법은 더 빠른 컴퓨터를 사용하는 것
- **딥러닝은 병렬 처리를 하는 계산 가속기인 GPU를 사용**
 - 개인이 쓰기에 적합한 GPU는 Nvidia GTX 등을 구매 가능
 - 캐글이나 코랩을 사용하는 사람들은 무료로 제공하는 GPU 또는 TPU를 사용할 수 있음

5.6 딥러닝 학습 전략

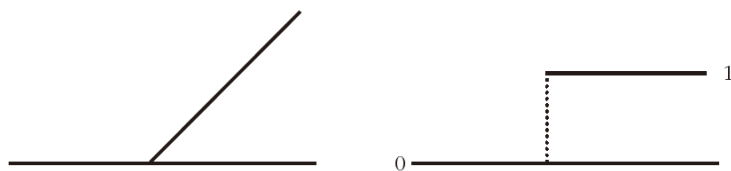
5.6.1.2 좋은 활성화함수 사용으로 해결 : ReLU 함수 사용

- 그래디언트 소멸 문제는 시그모이드 함수에서 더욱 심함
 - (x축) $S=5$ 이면 시그모이드의 그래디언트는 0에 가까움. 정확하게는 0.0000004501 임.



(a) tanh 시그모이드와 그래디언트

- ReLU 함수는 시그모이드 함수의 문제점을 해소해 줌
 - ReLU는 s 가 양수일 때 그래디언트가 1이고, 음수일 때 0임
 - 따라서 시그모이드에 비해 그래디언트 소멸이 발생할 가능성이 낮음



(b) ReLU와 그래디언트

5.6 딥러닝 학습 전략

5.6.2 과잉 적합과 과소 적합 회피 전략

- 과소 적합과 과잉 적합
 - 데이터에 비해 작은 용량의 모델을 사용해 오차가 많아지는 현상을 **과소 적합(underfitting)**이라고 함
 - 맨 왼쪽은 1차 방정식을 모델로 채택한 경우이며, 훈련데이터를 제대로 모델링하기에는 용량이 작다(가중치의 수가 작은 모델)는 사실을 쉽게 알 수 있음
 - 과소 적합을 해소하려면 모델의 용량을 크게 하면 됨
 - 아래의 그림은 용량이 큰 모델인 2차, 3차, 12차 방정식을 모델로 사용한 경우의 예시

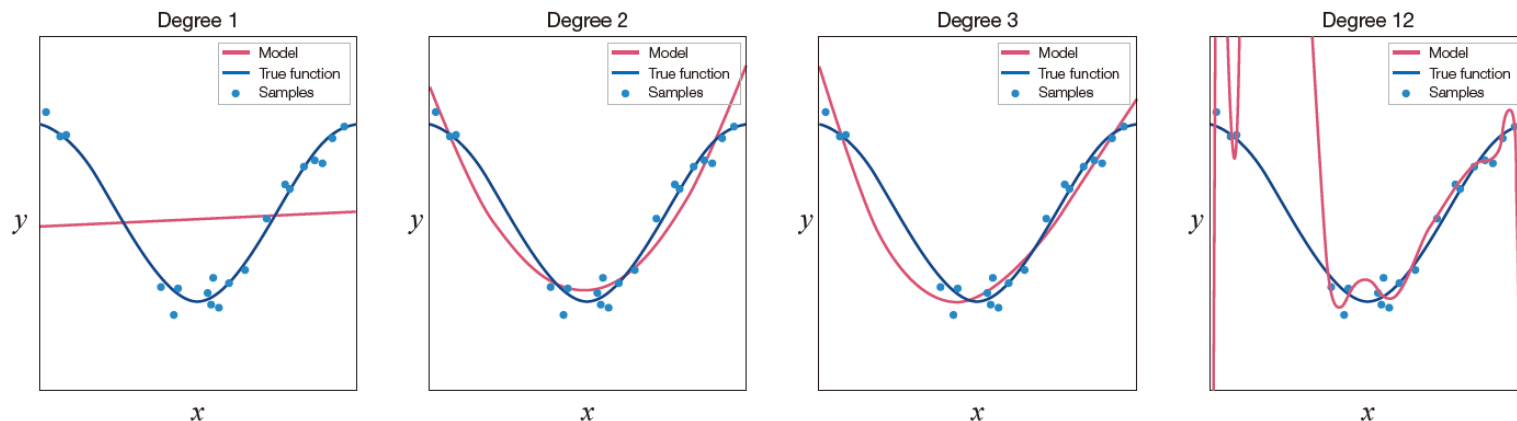


그림 5-12 과소 적합과 과잉 적합

5.6 딥러닝 학습 전략

5.6.2 과잉 적합과 과잉 적합 회피 전략

- 과소 적합과 과잉 적합
 - 〈너무 큰 모델〉이 데이터에 과도하게 적응하여 일반화 능력을 잃는 현상을 **과잉 적합 overfitting**이라 부름
 - 기계 학습의 최종 목적은 일반화 능력을 최대화 하는 것. 즉, 새로운 샘플에 대해 높은 정확률을 확보하는 것
 - 그런데, 빨간 점으로 표시된 예측 값이 파란 점으로 표시된 참값을 크게 벗어나는 결과를 보이고 있으며, 이는 모델의 용량이 데이터 복잡도에 비해 너무 커서 발생함
 - [그림 5-13]은 x_0 라는 새로운 샘플에 대해 **12차 모델**이 예측한 결과를 보여줌

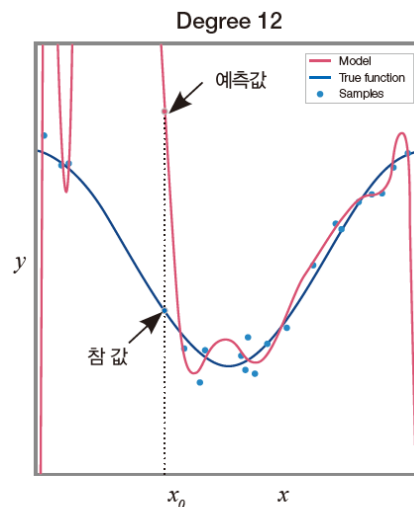


그림 5-13 과잉 적합에 따른 부정확한 예측

5.6 딥러닝 학습 전략

5.6.2 과잉 적합과 과잉 적합 회피 전략

- 과소 적합과 과잉 적합
 - 너무 큰 모델이 데이터에 과도하게 적응하여 일반화 능력을 잃는 현상을 **과잉 적합 overfitting** 이라 부름
- 과잉 적합을 해소하는 전략은 아주 많은데, 그중 가장 확실한 방법은 데이터의 양을 늘리는 것**

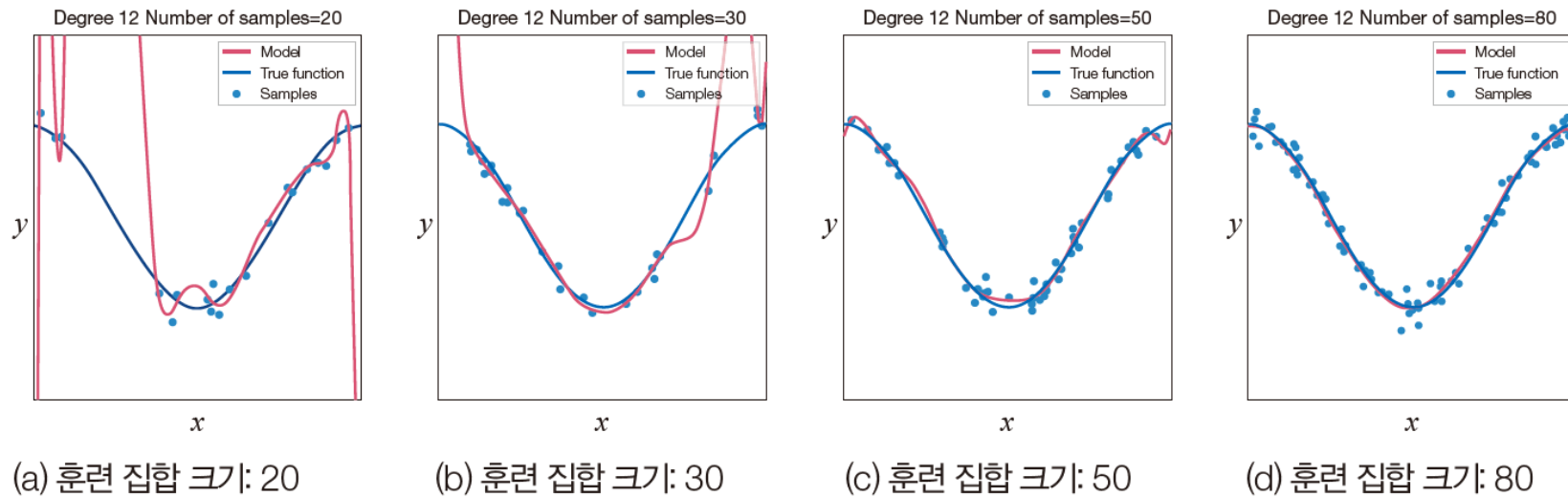


그림 5-14 데이터 양을 늘려 과잉 적합을 해소

5.6 딥러닝 학습 전략

5.6.2 과잉 적합과 과잉 적합 회피 전략

- 과소 적합과 과잉 적합
 - 너무 큰 모델이 데이터에 과도하게 적응하여 일반화 능력을 잃는 현상을 **과잉 적합 overfitting**이라 부름
 - **딥러닝은 큰 용량의 모델을 사용하되 적절한 규제 기법을 적용해 과잉 적합을 피하는 전략을 구사함**
 - 딥러닝에서 사용하는 규제 기법은 데이터의 양을 늘리는 것 외에 조기 멈춤, 가중치 감쇠, 드롭아웃, 앙상블 등이 있음 (6장에서 다룰 예정)

5.7 딥러닝이 사용하는 손실 함수

5.7.1 평균제곱오차

- 통계학에서 아주 오래 전부터 사용된 대표적인 손실 함수로서 신경망이 빌려 쓰는 것
 - 신경망은 여러 개의 샘플로 구성된 미니배치 단위로 손실 함수를 정의하며, 식(5.8)을 미니 배치 단위에 적용하고 평균을 취한 값이 평균제곱오차(MSE, mean squared error)에 해당 됨

$$e = \| \mathbf{y} - \mathbf{o} \|^2 \quad (5.8)$$

$$\begin{aligned} J(\mathbf{U}^1, \mathbf{U}^2, \dots, \mathbf{U}^L) &= \frac{1}{|M|} \sum_{\mathbf{x} \in M} \| \mathbf{y} - \mathbf{o} \|^2 \\ &= \frac{1}{|M|} \sum_{\mathbf{x} \in M} \| \mathbf{y} - \tau_L \left(\dots \tau_2 \left(\mathbf{U}^2 \tau_1 \left(\mathbf{U}^1 \mathbf{x}^T \right) \right) \right) \|^2 \end{aligned} \quad (5.6)$$

- 평균제곱오차의 문제점
 - 오차 e 가 더 큰데 그레디언트가 더 작은 상황이 발생하기도 함
 - 이는 공부를 못하는 학생이 더 높은 점수를 받는 상황(학습 의욕 저하)과 비슷하며, 학습이 느려지거나 학습이 안되는 상황을 초래할 가능성이 높음

5.7 딥러닝이 사용하는 손실 함수

5.7.2 교차 엔트로피

- 교차 엔트로피는 평균제곱오차의 불공정성 문제를 해결해 줌
- 엔트로피는 확률 분포의 무작위성^{randomness}, 즉 불확실성^{uncertainty}을 측정하는 함수임
 - 엔트로피는 새너이 제안한 정보 이론에서 유래함
- 엔트로피 예시
 - 공정한 주사위의 확률 분포는 1,2,3,4,5,6이 나올 확률이 모두 1/6이므로 불확실성이 가장 높음
 - 찌그러진 주사위가 있어 1의 면적이 가장 넓다면 (예측 가능성이 높아지며) 불확실성이 낮아짐
 - ➔ 엔트로피로 표현하면, 공정한 주사위의 엔트로피가 가장 높고, 많이 찌그러질수록 엔트로피가 낮아짐
- 엔트로피 식

$$H(x) = - \sum_{i=1,k} P(e_i) \log P(e_i) \quad (5.9)$$

5.7 딥러닝이 사용하는 손실 함수

5.7.2 교차 엔트로피

- 교차 엔트로피는 두 확률 분포가 다른 정도를 측정 (정답과 예측한 확률 분포의 다른 정도 측정에 활용)
- 교차 엔트로피 예시
 - 공정한 주사위 2개가 있을 때, 둘의 확률 분포가 같으므로 교차 엔트로피가 작음
 - 공정한 주사위와 찌그러진 주사위로 교차 엔트로피를 계산하면 큰 값이 됨
- 교차 엔트로피 식

$$H(P, Q) = -\sum_{i=1,k} P(e_i) \log Q(e_i) \quad (5.10)$$

공정한 주사위 P와 Q의 교차 엔트로피 (모두 1/6)

$$-\left(\frac{1}{6} \log \frac{1}{6} + \dots + \frac{1}{6} \log \frac{1}{6}\right) = 1.7918$$

공정한 주사위 P와 찌그러진 주사위 Q(1이 1/2, 나머지는 1/10 확률)의 교차 엔트로피

$$-\left(\frac{1}{6} \log \frac{1}{2} + \frac{1}{6} \log \frac{1}{10} + \dots + \frac{1}{6} \log \frac{1}{10}\right) = 2.0343$$

5.7 딥러닝이 사용하는 손실 함수

5.7.2 교차 엔트로피

- 교차 엔트로피는 평균제곱오차의 불공정성 문제를 해결해 줌
- 딥러닝은 손실 함수로 교차 엔트로피를 사용함
 - C: 부류의 개수
 - O: 신경망의 출력
 - Y: 레이블 (원핫인코딩)

$$\text{교차 엔트로피 손실 함수: } e = -\sum_{i=1,C} y_i \log o_i \quad (5.11)$$

[예제 5-1] 교차 엔트로피의 합리성 확인

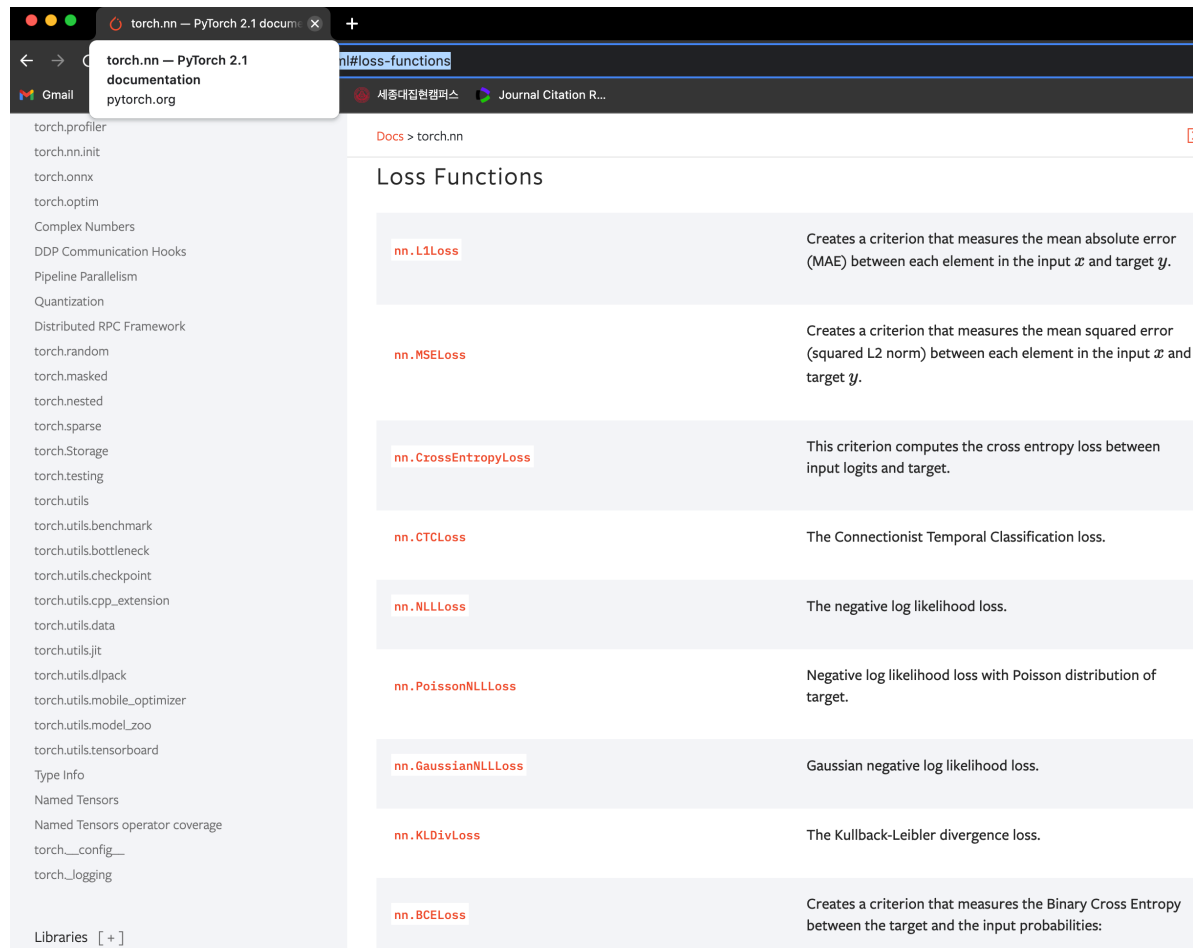
숫자 인식에서 부류 3에 속하는 샘플의 경우 $y=(0,0,0,1,0,\dots,0)$ 이다. 신경망의 출력이 $o=(0.1,0,0,0.9,0,\dots,0)$ 이라 하자. 부류 3에 해당하는 곳이 0.9로서 가장 크므로 신경망이 샘플을 맞힌 경우이다. 식 (5.11)을 계산하면 $e=-(0 \times \log(0.1)+0 \times \log(0)+0 \times \log(0)+1 \times \log(0.9)+\dots+0 \times \log(0))=0.1054$ 가 된다.

이제 신경망이 $o=(0,0.9,0,0.1,0,\dots,0)$ 을 출력했다고 가정하자. 부류 1에 해당하는 곳이 가장 큰 값을 갖기 때문에 신경망이 틀린 경우이다. 이 경우 $e=-(0 \times \log(0)+0 \times \log(0.9)+0 \times \log(0)+1 \times \log(0.1)+\dots+0 \times \log(0))=2.3026$ 이 된다. 후자의 틀린 경우에서 손실 함수 값이 훨씬 큰 사실을 확인할 수 있다.

5.7 딥러닝이 사용하는 손실 함수

5.7.3 손실 함수의 성능 비교 실험 (MNIST)

- 파이토치에서 지원하는 손실함수는 아래와 같이 확인 가능
 - <https://pytorch.org/docs/stable/nn.html#loss-functions>



5.7 딥러닝이 사용하는 손실 함수

5.7.3 손실 함수의 성능 비교 실험 (MNIST)

[프로그램5-10] 손실 함수의 성능 비교: 평균제곱오차와 교차 엔트로피

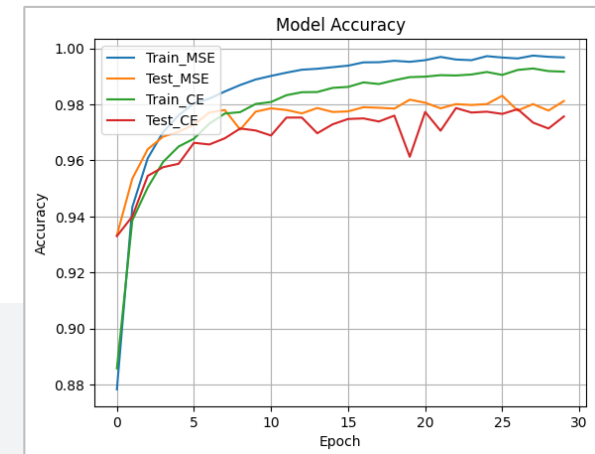
```
history_mse = dict()
history_mse['train_acc'] = list()
history_mse['test_acc'] = list()
history_mse['train_loss'] = list()
history_mse['test_loss'] = list()

for iter in tqdm.tqdm(range(30)):
    DNN, history_mse = train_epoch(train_dataloader, optimizer, DNN, loss_mse, history_mse)
    with torch.no_grad():
        DNN, history_mse = test_epoch(test_dataloader, optimizer, DNN, loss_mse, history_mse)

DNN = DeepNeuralNetwork(input_dim=784, hidden_dim=512, output_dim=10)
DNN = DNN.cuda()
optimizer = torch.optim.Adam(DNN.parameters(), lr=0.001)

history_ce = dict()
history_ce['train_acc'] = list()
history_ce['test_acc'] = list()
history_ce['train_loss'] = list()
history_ce['test_loss'] = list()

for iter in tqdm.tqdm(range(30)):
    DNN, history_ce = train_epoch(train_dataloader, optimizer, DNN, torch.nn.CrossEntropyLoss(), history_ce)
    with torch.no_grad():
        DNN, history_ce = test_epoch(test_dataloader, optimizer, DNN, torch.nn.CrossEntropyLoss(), history_ce)
```



08행 loss_mse 사용하기

10행 loss_mse 사용하기

23행 CrossEntropyLoss 사용하기

25행 CrossEntropyLoss 사용하기

5.8 딥러닝이 사용하는 옵티마이저

- 신경망 학습은 손실 함수의 최저점을 찾아가는 과정
- 최저점을 찾는 문제는 전형적인 최적화 문제
- 신경망에서 SGD와 같은 최적화 알고리즘을 옵티마이저 *optimizer* 라고 부름
- 신경망 학습에 이용되는 데이터는 잡음과 변화가 아주 심하여 SGD 옵티마이저가 종종 한계를 들어내며, 이런 한계 극복을 위해 모멘텀 *momentum*과 적응적 학습률 *adaptive learning rate* 이라는 방법이 소개됨

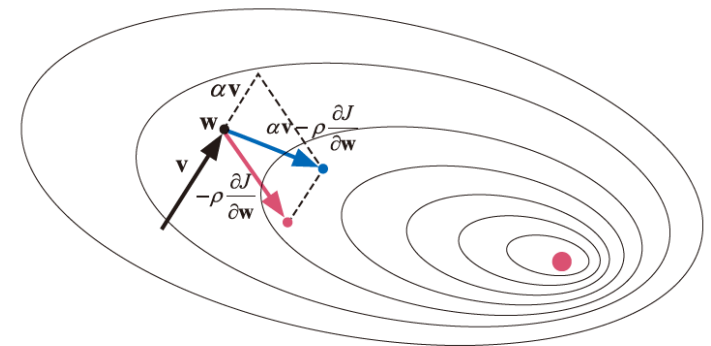
5.8 딥러닝이 사용하는 옵티마이저

5.8.1 모멘텀 momentum 을 적용한 옵티마이저

- 물리에서 모멘텀은 이전 운동량을 현재에 반영하는 아이디어를 뜻하며 관성과 관련이 깊음
- 모멘텀을 신경망에 적용하면 성능 향상이 뚜렷하게 나타남

5.8.1.1 모멘텀

- 검은색 실선 : 이전 미니배치에서 누적된 <방향 벡터gradient> v
- 검은 점: 현재 가중치 벡터 w
- 빨간 실선 : 현재 가중치 w 를 현재 미니배치에서 계산된 방향 벡터를 고려하여 빨간 점으로 이동
→ 고전적인 SGD
- 파란 실선 : 현재 가중치 w 를 현재 미니배치에서 계산된 방향 벡터와 이전 미니배치에서 누적된 방향 벡터를 함께 고려하여 파란 점으로 이동 → 모멘텀을 적용한 SGD

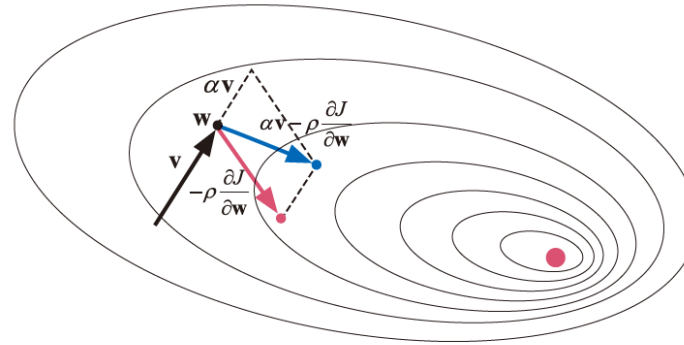


5.8 딥러닝이 사용하는 옵티마이저

5.8.1.1 모멘텀

$$\text{고전적 SGD: } \mathbf{w} = \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$$

$$\begin{aligned} \text{모멘텀을 적용한 SGD: } \mathbf{v} &= \alpha \mathbf{v} - \rho \frac{\partial J}{\partial \mathbf{w}} \\ \mathbf{w} &= \mathbf{w} + \mathbf{v} \end{aligned}$$



- $\alpha=0$ 는 고전적 SGD, 모멘텀을 적용한 SGD α 는 $[0,1]$ 사이에서 조절
- α 가 1에 가까울수록 이전 정보에 큰 가중치 부여
- 보통 $\alpha=0.5, 0.9$ 를 사용

5.8 딥러닝이 사용하는 옵티마이저

5.8.2 적응적 학습률을 적용한 옵티마이저

- 고전적 SGD 옵티마이저에는 학습률 learning rate 이라는 하이퍼 매개변수가 존재

$$\text{고전적 SGD: } \mathbf{w} = \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$$

- 그래디언트는 오류가 작아지는 방향을 지시하지만, 얼마나 이동해야 최저점에 도달하는지 정보 부재
- 따라서 SGD 옵티마이저는 작은 학습률을 사용해 조금씩 이동하는 보수적인 정책을 사용
- 일반적으로 SGD는 학습률을 0.01을 기본값으로 사용하는 편인데 더 작은 값을 사용하는 경우도 많음

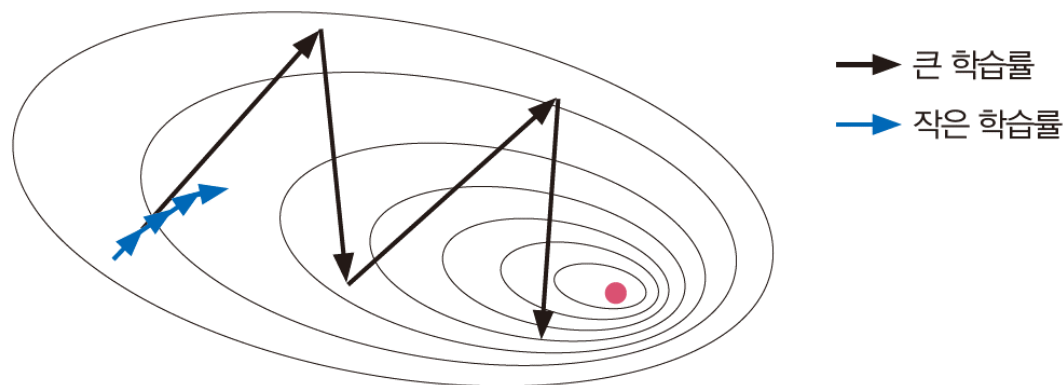


그림 5-16 학습률에 따른 수렴 특성

5.8 딥러닝이 사용하는 옵티마이저

5.8.2 적응적 학습률을 적용한 옵티마이저

- 고전적 SGD는 아래와 같이 고정된 학습률을 사용하지만, 상황에 맞게 학습률을 조절하는 적응적 학습률이 존재함

$$\text{고전적 SGD: } \mathbf{w} = \mathbf{w} - \rho \frac{\partial J}{\partial \mathbf{w}}$$

- 적응적 학습률을 적용한 옵티마이저에는 Adagrad, RMSprop, Adam 등이 있음
 - Adagrad : 이전 그래디언트를 누적한 정보를 이용하여 학습률을 적응적으로 설정하는 기법
 - RMSprop : 이전 그래디언트를 누적할 때 오래된 것의 영향 ρ 을 줄이는 정책을 사용하여 AdaGrad를 개선한 기법
 - Adam : RMSProp에 모멘텀 β_1 을 적용하여 RMSprop β_2 를 ρ 대신 사용을 개선한 기법

5.8 딥러닝이 사용하는 옵티마이저

5.8.3 옵티마이저 성능 비교 실험 (Fashion MNIST)

➔ 깊은 다층 퍼셉트론은 프로그램 [5-10]과 동일

[프로그램5-11] 옵티마이저의 성능 비교: SGD, Adam, Adagrad, RMSprop

```
train_dataloader = torch.utils.data.DataLoader(training_data, batch_size=128, shuffle=True)
test_dataloader = torch.utils.data.DataLoader(test_data, batch_size=128, shuffle=False)

for optimizer_name in ['SGD', 'AdaGrad', 'RMSprop', 'Adam']:
    DNN = DeepNeuralNetwork(input_dim=784, hidden_dim=512, output_dim=10)
    DNN = DNN.cuda()
    if optimizer_name == 'SGD':
        optimizer = torch.optim.SGD(DNN.parameters(), lr=0.001)
    elif optimizer_name == 'AdaGrad':
        optimizer = torch.optim.Adagrad(DNN.parameters(), lr=0.001)
    elif optimizer_name == 'RMSprop':
        optimizer = torch.optim.RMSprop(DNN.parameters(), lr=0.001)
    elif optimizer_name == 'Adam':
        optimizer = torch.optim.Adam(DNN.parameters(), lr=0.001)

    history[optimizer_name]['train_acc'] = list()
    history[optimizer_name]['test_acc'] = list()
    history[optimizer_name]['train_loss'] = list()
    history[optimizer_name]['test_loss'] = list()

    for iter in tqdm.tqdm(range(30)):
        DNN, history[optimizer_name] = train_epoch(train_dataloader,
                                                    optimizer, DNN, loss_mse, history[optimizer_name])

    with torch.no_grad():
        DNN, history[optimizer_name] = test_epoch(test_dataloader,
                                                    optimizer, DNN, loss_mse, history[optimizer_name])
```

08행 SGD 옵티마이저 사용하기

10행 AdaGrad 옵티마이저 사용하기

12행 RMSProp 옵티마이저 사용하기

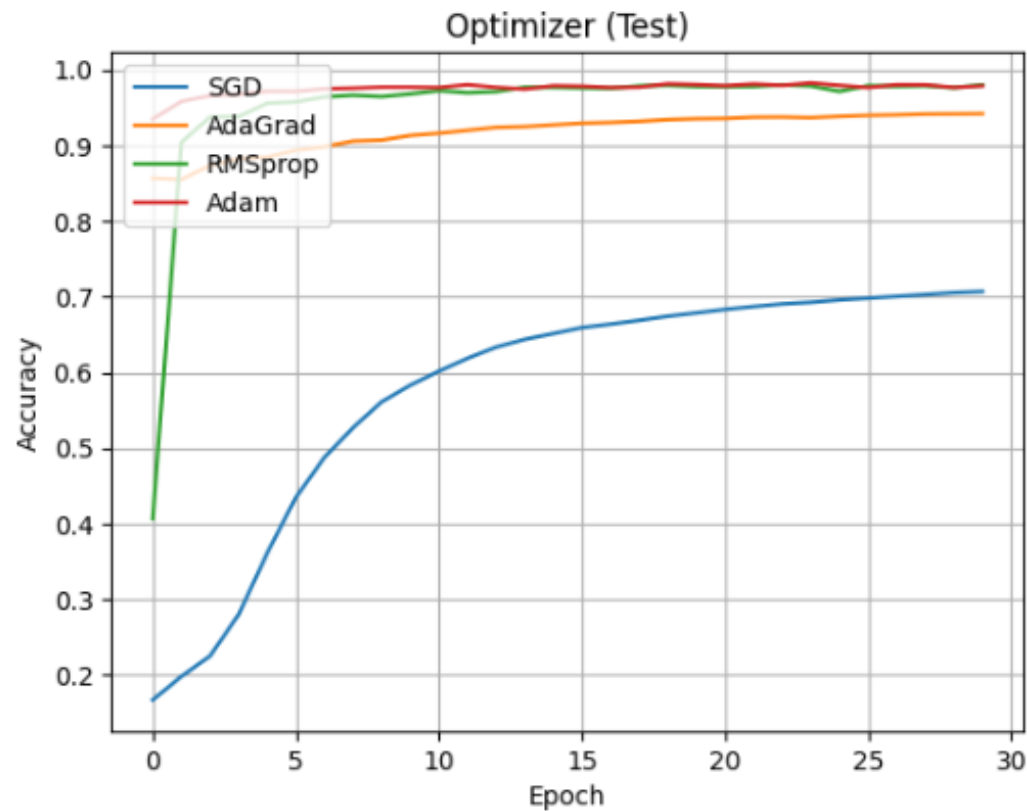
14행 Adam 옵티마이저 사용하기

5.8 딥러닝이 사용하는 옵티마이저

5.8.3 옵티마이저 성능 비교 실험 (Fashion MNIST)

➔ 깊은 다층 퍼셉트론은 프로그램 [5-10]과 동일

[프로그램5-11] 옵티마이저의 성능 비교: SGD, Adam, Adagrad, RMSprop



5.9 좋은 프로그래밍 스킬

- 프로그래밍 스킬의 중요성
 - 프로그래밍을 못하면 아무런 인공지능 프로그램도 만들 수 없음
 - 프로그래밍이 미숙하면 좋은 아이디어보다 디버깅에 에너지 소진
 - 프로그래밍은 인공지능에서 충분조건은 아니지만 핵심 필요조건
- 모듈화 하라
 - 좋은 프로그램을 짜기 위해 가장 먼저 할 일은 함수를 만들어 반복되는 코드를 줄이는 것
- 언어의 좋은 특성을 최대한 활용하라
 - 사례(②의 두 행을 간결하게 ①의 한 행으로 코딩)

```
print("SGD 정확률은",dmlp_sgd.evaluate(x_test,y_test,verbose=0)[1]*100)
```

①

```
res_sgd=dmlp_mse.evaluate(x_test,y_test,verbose=0)  
print("평균제곱오차의 정확률은",res_sgd[1]*100)
```

②

5.9 좋은 프로그래밍 스킬

- 점증적으로 코딩하라
 - 한번에 한가지 기능을 추가하고 옳게 작동하는지 확인하는 일을 반복
- 디자인 패턴이 몸에 배게 하라
 - 다른 프로그램과 공유하는 디자인 패턴에 대한 눈썰미
- 도구에 한없이 익숙해져라
 - 라이브러리 사용에 익숙
 - 통합개발환경 사용에 익숙
- 기초에 충실하라
 - 파이썬 기초 문법 익히기
 - 중요한 라이브러리 익히기
 - 기계학습의 기초 이론 등

5.10 교차 검증을 이용한 하이퍼 매개변수 최적화

- 높은 성능을 발휘하는 모델을 찾으려면 우연을 배제할 수 있는 객관적인 성능 평가가 필수임
- 가장 효과적인 방법은 여러 번 반복하고 평균을 취하는 교차 검증임

5.10.1 교차 검증을 이용한 옵티마이저 선택

- 딥러닝 라이브러리 (파이토치, 텐서플로우) 에는 교차 검증을 지원하는 클래스가 없음
- 딥러닝은 주로 대용량 데이터셋으로 수행하므로 교차 검증 없이 측정한 성능도 어느 정도 신뢰할 수 있다는 생각 때문임
- 따라서 필요시 교차검증을 하는 함수를 직접 만들어야 함

5.10 교차 검증을 이용한 하이퍼 매개변수 최적화

5.10.1 교차 검증을 이용한 옵티 마이저 성능 선택

[프로그램5-12] K-Fold 교차 검증을 활용한 옵티마이저의 성능 비교

```
118 # Dataset을 slicing하기 위해 Tensor로 변환
119 x_train = training_data.data
120 y_train = training_data.targets
121 x_train = x_train.type(torch.float32)
```

119-121행 K-Fold slicing을 위해 Tensor로 자료형 변경

```
122 kfold_histroy = list()
```

```
123
124
125 for train_index, val_index in KFold(k).split(x_train):
```

125행 K-Fold 검증을 위해 k번 돌아가는 반복문 정의

```
126
127     history = dict()
128     history['SGD'] = dict()
129     history['AdaGrad'] = dict()
130     history['RMSprop'] = dict()
131     history['Adam'] = dict()
```

```
132
133     xtrain, xval = x_train[train_index], x_train[val_index]
134     ytrain, yval = y_train[train_index], y_train[val_index]
```

133-137행 K-Fold 검증을 위해 train/val 데이터 정의

```
135
136     train_dataloader = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(xtrain,ytrain),batch_size=128,shuffle=True)
137     val_dataloader = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(xval,yval),batch_size=128,shuffle=False)
```

```
138
139     for optimizer_name in ['SGD', 'AdaGrad', 'RMSprop', 'Adam']:
140         DNN = DeepNeuralNetwork(input_dim=784, hidden_dim=512, output_dim=10)
141         DNN = DNN.cuda()
142         if optimizer_name == 'SGD':
143             optimizer = torch.optim.SGD(DNN.parameters(), lr=0.001)
144         elif optimizer_name == 'AdaGrad':
145             optimizer = torch.optim.Adagrad(DNN.parameters(), lr=0.001)
146         elif optimizer_name == 'RMSprop':
147             optimizer = torch.optim.RMSprop(DNN.parameters(), lr=0.001)
148         elif optimizer_name == 'Adam':
149             optimizer = torch.optim.Adam(DNN.parameters(), lr=0.001)
```

```
150
151     history[optimizer_name]['train_acc'] = list()
152     history[optimizer_name]['test_acc'] = list()
153     history[optimizer_name]['train_loss'] = list()
154     history[optimizer_name]['test_loss'] = list()
```

```
155
156     for iter in tqdm.tqdm(range(5)):
157         DNN, history[optimizer_name] = train_epoch(train_dataloader,
158                                                     optimizer, DNN, loss_mse, history[optimizer_name])
159         with torch.no_grad():
160             DNN, history[optimizer_name] = test_epoch(val_dataloader,
161                                                         optimizer, DNN, loss_mse, history[optimizer_name])
162     kfold_histroy.append(history)
```

162행 K-Fold 검증을 위해 k별 validation 정확도 기록

5.10 교차 검증을 이용한 하이퍼 매개변수 최적화

5.10.1 교차 검증을 이용한 옵티 마이저 성능 선택

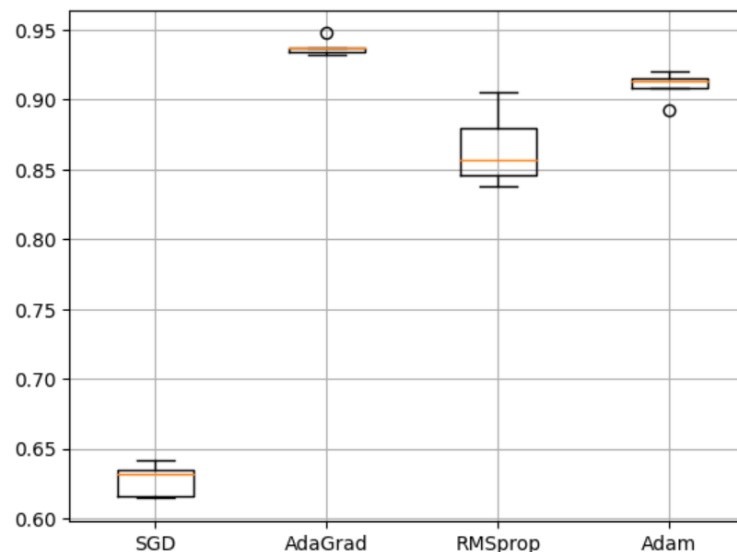
[프로그램5-12] K-Fold 교차 검증을 활용한 옵티마이저의 성능 비교

```
165 test_acc_dict = dict()
166 test_acc_dict['SGD'] = list()
167 test_acc_dict['AdaGrad'] = list()
168 test_acc_dict['RMSprop'] = list()
169 test_acc_dict['Adam'] = list()
170
171 for optimizer_name in ['SGD', 'AdaGrad', 'RMSprop', 'Adam']:
172     for idx in range(len(kfold_histroy)):
173         test_acc_dict[optimizer_name].append(kfold_histroy[idx][optimizer_name]['test_acc'][-1])
174
175
176 # 박스 플롯으로 정확도를 표시
177 plt.grid()
178 plt.boxplot([test_acc_dict['SGD'], test_acc_dict['AdaGrad'], test_acc_dict['RMSprop'], test_acc_dict['Adam']], labels=['SGD', 'AdaGrad', 'RMSprop', 'Adam'])
```

165-169행 각각의 K 마다 마지막 epoch에 대한 정확도를 담아줄 list 정의

171-173행 옵티마이저 별 validation 정확도 저장

177행 boxplot을 활용해 K-Fold 교차 검증 시각화



5.10 교차 검증을 이용한 하이퍼 매개변수 최적화

5.10.2 과도한 계산 시간과 해결책

- 교차 검증은 시간이 많이 소요됨, 즉 성능 검증에 대한 신뢰도가 높아지는 대신 시간이 걸림
 - 학습을 한번 마치는데 t 라는 시간이 걸린다면, kt 만큼 시간이 지나야 옵티마이저 하나의 성능 측정이 끝남
 - [프로그램5-12]는 옵티마이저 4개를 평가하므로 총 $4kt$ 만큼의 시간이 걸림
 - [프로그램5-12]는 하이퍼 매개변수를 모두 고정시키고 최적화를 수행
- 딥러닝에서 만일 하이퍼 매개변수 최적화를 제대로 하려면 훨씬 많은 시간이 걸림
 - 데이터셋의 크기가 기존 토이 데이터셋보다 훨씬 큼
 - 딥러닝은 하이퍼 매개변수가 아주 많음
 - 예를들어, 옵티마이저 {SGD, Adam, Adagrad, RMSprop}, 학습률 {0.01, 0.005, 0.001, 0.0005, 0.0001, 0.00005, 0.00001}, 미니배치 크기 {32, 64, 128, 256, 512, 1024}의 조합을 최적화한다면 총 168개 조합에 대해서 학습을 진행해야 함
- 딥러닝은 GPU를 이용함으로써 과도한 계산 시간 문제를 해결 가능함